Empirical Software Engineering (2026) 31:185
https://doi.org/10.1007/s10664-026-10915-2

Smelly-shot is all you need: an empirical study to compare in-context learning verses fine tuning for code smell detection

Nawaf Alomari[1] · **Moussa Redah**[1] · **Ahmad Ashraf**[1] · **Mohammad Alshayeb**[1,2]

Received: 4 March 2025 / Accepted: 23 June 2026

Abstract

Context Code smells, such as God Class or Data Class, are poor design patterns that negatively impact software quality, particularly maintainability. Large Language Models (LLMs) have demonstrated success across various domains by employing fine-tuning or in-context learning (ICL) to perform specific tasks.

Objective This study aims to empirically compare the effectiveness and efficiency of fine-tuning and ICL in code smell detection tasks. Effectiveness is measured using the F1 Score, while efficiency is assessed in terms of training time and data usage.

Method The experiments were conducted using three datasets: MLCQ, CoRT, and an artificially constructed dataset. For ICL, models GPT4oMini, Gemini-flash, and CodeLLaMA70B, CodeGPT, and GPT-2 were tested with 0, 1, 3, and 5-shot settings. For fine-tuning, CodeBERT, GraphCodeBERT, and UniXcoder, CodeGPT, and GPT-2 were utilized, and hyperparameter tuning was done. The two compact models, CodeGPT and GPT-2, were applied to both techniques, enabling a controlled comparison in which only the learning technique is changed.

Results Fine-tuning consistently outperformed ICL, achieving a mean F1 Score improvement of 26%. However, ICL demonstrated greater efficiency, requiring fewer training samples (a mean difference of 1041.25 samples) and less training time (a mean difference of 442.57 seconds). Furthermore, ICL exhibited better generalization capabilities and more flexible adaptation to changes.

Conclusions Fine-tuning is recommended when labeled data is abundant, and high performance is critical, especially for domain-specific datasets. In contrast, ICL is preferable in scenarios with insufficient labeled data, such as for low-resource languages or when a more general-purpose model is needed. Additionally, ICL is advantageous when computational resources and time are limited.

Keywords Code smell detection · LLM · Fine-tuning · In-context learning

Communicated by: Ali Ouni.

Extended author information available on the last page of the article

1 Introduction

Software systems play an integral role in modern life. As these systems become increasingly essential, the demand for their development continues to rise. This growing demand is often accompanied by high costs and tight deadlines, leading to the accumulation of technical debt and the emergence of code smells. Code smells refer to poor design choices that can potentially introduce more defects in future software releases. Studies have demonstrated that code smells degrade the quality of both the code and the software systems (Fontana et al. 2013). With these large software systems, the cost of maintenance is higher than the production (Banker et al. 1993). So, having a code smell will lead to an increase in the cost of the system.

The detection of code smells is critical for ensuring software quality. However, with codebases' increasing size and complexity, manual detection becomes impractical, necessitating automated solutions. Traditional approaches for detecting code smells have primarily relied on using structural parameters to construct detection rules (Moha et al. 2009), or on metric thresholds (Bafandeh Mayvan et al. 2020; Chen et al. 2018; Sharma and Chhabra 2025) to detect code smells. However, such approaches are highly dependent on the thresholds they adopt, and there is no universal agreement on what these thresholds should be Lacerda et al. (2020). In contrast, the use of large language models can leverage both the structural and semantic information embedded in source code, capturing richer code representations(Alazba et al. 2024) . In the same manner, numerous studies have developed automated tools leveraging machine learning algorithms and other techniques to identify code smells (Hamdy and Tazy 2020; Li et al. 2020; AbuHassan et al. 2021; Alazba et al. 2023). More recently, fine-tuning pre-trained language models has emerged as a promising approach for detecting code smells (Alazba et al. 2024). While effective, these techniques demand large volumes of labeled data and significant computational resources. As large language models (LLMs) grow in complexity, they inherently encapsulate sufficient knowledge to perform tasks like code smell detection. The challenge lies in formulating these tasks to activate the model's capabilities, a concept known as in-context learning (ICL) (Radford et al. 2019).

Extensive research has been conducted to explore ICL and compare it to fine-tuning across various domains (Mosbach et al. 2023; Yin et al. 2024). Furthermore, some studies have investigated prompting techniques and ICL for code smell detection (Chen et al. 2024; Zhang and Saber 2024; Silva et al. 2024; Baumgartner et al. 2024; Wu et al. 2024). However, many of these studies lack a clear systematic approach. Variations in the number of shots (examples) used during ICL were not thoroughly examined. Additionally, some studies failed to explicitly report the exact LLM or its version, with vague references such as "ChatGPT," which does not identify a specific model. Moreover, empirical comparisons between ICL and fine-tuning are often absent for code smell detection, leaving critical questions unanswered.

ICL offers a promising alternative for code smell detection, especially as supervised fine-tuning requires extensive labeled datasets and suffers from generalizability issues (Liu et al. 2021). Motivated by these gaps, this work aims to empirically compare ICL versus fine-tuning for code smell detection, analyzing their limitations and advantages compared to each other. The contributions of this work are as follows, with detailed research questions (RQs) and their motivations outlined in Table 1:

Table 1 Research questions and motivations

Research Question	Motivation
RQ1: Which technique achieves better performance in the task?	This helps identify the technique to use when the goal is to achieve the highest performance.
RQ2: Which technique can generalize better to other datasets?	This helps identify which technique generalizes better, determines the appropriate technique for different situations, and which technique can be preferred in domain-specific datasets.
RQ3: Which of these two techniques is preferred based on the availability of data?	This question will help identify whether in-context learning or fine-tuning is more suitable when datasets are limited.
RQ4: Which of these two techniques is preferred based on the available hardware resources and time?	This helps identify the technique to use when computational power or time is constrained.

- **Empirical comparison of ICL and fine-tuning performance:** Evaluate the effectiveness of ICL and fine-tuning for code smell detection tasks (H_0^1 and RQ1).
- **Analysis of data consumption:** Examine how each technique consumes data, considering the inherent nature of the respective methods (H_0^3 and RQ3).
- **Evaluation of preparation time:** Assess the time required to prepare each solution for inference, as it is crucial to understand how quickly a solution can be deployed (H_0^2 and RQ4).
- **Generalizability analysis:** Compare the generalizability of the two techniques and determine the contexts in which each is preferable (RQ2).

To define the scope of this experiment and establish a clear objective, we followed the **GQM** method. The primary objective of this experiment is to evaluate the **effectiveness** and **efficiency** of using **in context learning (ICL)** versus **fine tuning** for code smell detection for the **purpose of comparison** in terms of **number of training samples measured in number of samples, training time measured in seconds, performance measured in F1 score** from the point of view of **researchers** in the context of **8 large language models and 3 code smells dataset**.

We followed the guidelines proposed by Jedlitschka et al. (2008) for reporting experiments in software engineering. The rest of this paper is structured as Section 2 Background for ICL, fine-tuning, and code smell detection. Section 3 shows the related work. Section 4 shows the followed methodology. Then, in Section 5, the experiment is discussed. The results are shown in Section 6. The discussion and implications are shown in Section 7. Section 8 shows the identified threats to validity, and Section 9 concludes the research with future work.

2 Background

In this section, the common terminology used in this research is explained. Specifically, code smells and refactoring are introduced. Then, we present a brief overview of LLMs and how they can be tuned to follow tasks with fine-tuning and ICL.

2.1 Code smells

Code smells refer to potential design issues that may not cause immediate errors but can negatively affect external quality attributes, particularly maintainability (Yamashita and Counsell 2013). They often arise when programmers take shortcuts instead of following best practices. For example, consider a program that starts with limited functionality, leading to the creation of a single class. As more functionality is added over time, this class grows excessively large, taking on multiple responsibilities and becoming what is known as a *God Class*.

Code smells can manifest at different levels of code granularity, and some of the code smells and their descriptions are available in Table 2.

Refactoring is a key technique for addressing code smells. It involves restructuring the code to improve its design and maintainability while preserving its external behavior (Fowler 2018). Alshayeb (Alshayeb 2011) investigated the impact of refactoring on quality attributes by measuring changes before and after applying specific techniques, finding no universal improvement or consistent correlation. Building on this, Elish and Alshayeb (2011) classified refactoring techniques based on their effects on quality attributes, providing developers with a framework for selecting refactoring methods to target specific improvements.

2.2 LLMs and following tasks

LLMs are trained on vast amounts of text data to learn the statistical patterns of language, enabling them to predict the next token in a sequence. However, these models do not inherently understand or directly follow tasks; instead, they generate tokens based on the probability of their occurrence given the input tokens. LLMs can be fine-tuned for specific tasks such as text classification or question answering (Howard and Ruder 2018). There are two main approaches to enable LLMs to perform specific tasks:

Table 2 Common code smells and their descriptions

Code Smell	Description
God Class (Fowler 2018)	A class that takes on too many responsibilities, violating the Single Responsibility Principle, and should be divided into smaller, more focused classes.
Data Class (Fowler 2018)	A class that primarily consists of data fields without significant associated behavior or methods.
Long Method (Fowler 2018)	A method that is excessively long or complex, potentially benefiting from decomposition into smaller, more manageable methods.
Feature Envy (Fowler 2018)	A method that frequently accesses the data fields of another class instead of its own, indicating that the logic might belong in the accessed class.
Code Clone (Fowler 2018)	Identical or near-identical code fragments that are duplicated across multiple locations, leading to maintainability challenges.
Long Parameter List (Fowler 2018)	A method or function that takes an excessive number of parameters, making it difficult to understand and use, often indicating a need for better data grouping or encapsulation.

Fine-tuning This technique involves training the LLM on a task-specific dataset that contains input-output pairs. The model is trained to learn that, given a specific input, it should produce the corresponding output (Dodge et al. 2020). While widely used, fine-tuning is computationally expensive, especially for large-scale language models.

Prompting techniques Prompting allows LLMs to perform tasks without additional fine-tuning by structuring the input in a way that the model can interpret. Some prominent prompting techniques include:

- **In-Context Learning:** Proposed by Radford et al. (2019), this technique leverages the complexity of LLMs to interpret tasks formulated within the input context. The model performs the task by simply understanding the context provided.
- **Zero-Shot Learning:** A type of in-context learning where the task is described without providing any examples. The model is expected to infer the solution solely based on the task description (Radford et al. 2019).
- **Few-Shot Learning:** Introduced by Brown (2020), this method extends zero-shot learning by including a few demonstration examples alongside the task description. This approach has been shown to improve prediction performance.
- **Chain of Thought (CoT):** Proposed by Wei et al. (2022), this technique encourages the model to reason through problems step-by-step by using prompts such as "Let's think step by step." This has demonstrated improved performance across various tasks.

3 Related work

In this section, we review the related work for code smell detection with language models using two main approaches: Fine-tuning and ICL. The details are provided in the following subsections, while the summary is shown in Table 3.

3.1 Fine-tuning for code smell detection

Recent research has addressed the challenge of reliance on labeled datasets by introducing CoRT (Alazba et al. 2024) (Code Representations with Transformers) as a self-supervised learning framework for code smell detection. CoRT does not require manual annotation because it pre-trains on unlabeled source code and learns semantic and structural properties through a masked reserved word prediction challenge. By fine-tuning these representations, some code smells, including Data Class, God Class, Feature Envy, and Long Method, can be effectively detected. With high F1 scores (e.g., $> 88\%$) and great generalization abilities across unseen datasets, CoRT outperformed other deep learning models and conventional supervised approaches, proving to be a reliable and effective solution for code smell detection.

The difficulty of fine-tuning pre-trained models on small software engineering datasets has been brought to light by recent work, especially for tasks like code smell detection. Liu et al. (2023), developed the Variational Information Bottleneck (VIB) technique to solve the instability and overfitting problems that arise while fine-tuning big models like CodeBERT on limited datasets. Through the preservation of relevant task information, while compress-

Table 3 Summary of studies on code smell detection techniques

Ref	Technique	Smells Detected	Language
Alazba et al. (2024)	Fine-tuning	Data Class, God Class, Feature Envy, Long Method	Java
Liu et al. (2023)	Fine-tuning	Linguistic Code Smells	Java
Zhang et al. (2023)	Fine-tuning	Brain Class, Data Class, God Class, Brain Method	Java
Alrashedy and Binjahlan (2023)	Fine-tuning	General Code Smells	Python
Bhave and Sinha (2022)	Fine-tuning	Parameter List, Switch Statement	Java
Kovačević et al. (2022)	Fine-tuning	Long Method, God Class	Java
Sharma et al. (2021)	Fine-tuning	Feature Envy, Multi-faceted Abstraction	C#, Java
Liu et al. (2019)	Fine-tuning	Feature Envy, Long Method, Large Class, Misplaced Class	Java
Ren et al. (2021)	Fine-tuning	God Class	Java
Wang et al. (2020)	Fine-tuning	Feature Envy	Java
Chen et al. (2024)	In-context	Code anomalies (errors, flaws, vulnerabilities)	Python
Zhang and Saber (2024)	In-context	Code clone	Java
Silva et al. (2024)	In-context	Blob, Data Class, Feature Envy, Long Method	Java
Baumgartner et al. (2024)	In-context	Data Clumps	General (not specified)
This work	In-context and Fine-tuning	Feature Envy, Long Method, Large Class, Long Parameter List.	Java Mostly, Python, C++, JavaScript

ing irrelevant features in the intermediate embeddings, VIB enhances fine-tuning. In terms of stability and generalization, VIB continuously outperformed conventional fine-tuning, dropout, and weight decay when tested on tasks such as linguistic code scent detection. Its ability to improve the performance of pre-trained models on tiny datasets is demonstrated here, offering a strong foundation for software engineering jobs of a modest scale.

To improve detection performance, Zhang et al. (2023) introduced SCSmell, a code smell detection framework that combines pre-training with a three-layer stacking model. The technique combines textual data from a refined BERT pre-trained model with structural features obtained using static analysis methods. These characteristics are used to create Brain Class, Data Class, God Class, and Brain Method, which are labeled datasets for identifying code smells. According to experimental data, SCSmell performed better than

deep learning and conventional machine learning techniques, obtaining greater F1 scores, accuracy, precision, and recall. This study demonstrates how well pre-trained models may be adjusted to catch textual subtleties for better code smell detection.

In recent research, Alrashedy and Binjahlan (2023) introduced a novel approach for bug detection, which involves fine-tuning large pre-trained language models, such as CodeBERT and CodeT5, on real-world datasets. Their work highlights the challenges of directly fine-tuning these models on small datasets due to poor performance. To address this, they employed a multi-stage fine-tuning strategy, first training the models on a synthetic dataset before further fine-tuning on real-world data. This method significantly improved accuracy and F1 scores, demonstrating the potential of fine-tuned pre-trained models for tasks like code smell detection. The study underscores the importance of tailored fine-tuning strategies to enhance model performance on complex real-world software engineering problems.

Bhave and Sinha (2022) to identify two distinct code smells: switch statements and long parameter lists. The method combines structural and semantic information from code by integrating Fully Connected Networks (FCN) with a transformer model called DistilBERT. The model concatenates textual features (such as class and method names) with structural metrics (like coupling intensity and cyclomatic complexity) using embeddings produced by DistilBERT. The suggested design beats conventional models, such as CNN-BiLSTM pipelines and TF-IDF with Random Forest, achieving an accuracy of 91.2%. This study highlights the value of semantic and structural integration for enhancing classification performance and the efficacy of merging transformer-based language models with multimodal learning approaches for code smell detection.

Kovačević et al. (2022) suggested using pre-trained neural source code embeddings for the detection of God Class and Long Method code smells. The study tested embeddings such as code2vec, code2seq, and CuBERT. The results showed that CuBERT performed the best, with F-measures of 0.75 for Long Method detection and 0.53 for God Class. The paper investigates the benefits of embeddings for automatic feature extraction over conventional metric-based heuristics and draws attention to the difficulties associated with inconsistent class-level annotations using the MLCQ dataset, a large-scale manually labeled benchmark. This work represents a major advancement in the use of deep learning methods for precise and scalable code smell detection.

Another study by Sharma et al. (2021) explored the application of deep learning and transfer learning for detecting code smells in software systems. By employing Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and AutoEncoder models, the study evaluated their performance by identifying both implementation and design smells, such as complex methods and feature envy. Also, transfer learning was examined by training models on C# code and evaluating them on Java. The findings demonstrated the feasibility of both direct learning and transfer learning, with performance varying across smells and model types. In addition, the study emphasized the potential of deep learning methods in advancing code smell detection without extensive feature engineering, particularly for languages lacking robust smell detection tools. The open-source resources provided aim to foster replication and further research in this domain.

Wang et al. (2020) introduced a novel deep learning-based model for detecting the "Feature Envy" code smell using Bi-LSTM with a self-attention mechanism. The model integrates semantic distance information from method and class names (textual features) and structural distance metrics (numerical features) to identify misplaced methods that should

be refactored. The self-attention mechanism, paired with specially devised attention functions, improves the model's ability to capture semantic relationships. Evaluated on seven open-source Java projects, the approach outperformed traditional heuristic methods and prior deep learning models in smell detection and refactoring destination recommendation tasks. The study highlighted the potential of attention mechanisms for enhancing code smell detection accuracy and scalability in software engineering., This research underscores the potential of neural networks in advancing software refactoring practices. Liu et al. (2019) proposed a deep learning based framework for detecting code smells, exploiting automatically generated labeled datasets and neural network classifiers. The study introduced "smell-introducing refactorings" to integrate positive‘ training data by intentionally creating code smells in high-quality software, while negative samples were extracted directly from well-designed code. The proposed framework was evaluated on four types of smells: feature envy, long method, large class, and misplaced class. Experimental results demonstrated significant improvements in detection accuracy compared to existing tools; for misplaced class detection, they achieved an F-measure that increased up to 48.18%. The approach highlighted the scalability and adaptability of deep learning techniques in automated smell detection, reducing the limitations of manual feature engineering and heuristic methods.

Ren et al. (2021) introduced a new method for God Class detection showing multi-aspect interactions (code metrics, text features, and their relationships) and dataset fine-tuning. The Pre-training on synthetic datasets and fine-tuning with authentic ones addressed data quality and scarcity issues. The approach achieved significant improvements, with an F1 score of 34.19%, outperforming benchmarks like JDeodorant and prior deep learning methods. This study emphasizes the utility of fine-tuning and interaction modeling for accurate code smell detection in real-world scenarios.

3.2 In context learning code smell detection

Chen et al. (2024) looked into how to identify and fix code anomalies, including mistakes, flaws, and vulnerabilities, using large language models (LLMs) like OpenAI Codex, GPT-3, and GitHub Copilot. The significance of in-context learning was highlighted in the study, where models used specific inputs–prompts and code snippets–instead of keywords to improve context, leading to noticeably higher accuracy and F1-scores. Their results showed that LLMs could successfully detect and handle code anomalies when directed by thoughtfully created prompts. However, the study also pointed out drawbacks, like the requirement for large labeled datasets for supervised anomaly detection, which shows how scalable this method is. This study emphasizes how in-context learning might improve LLM's flexibility and accuracy when handling challenging code quality jobs.

The use of ChatGPT for identifying code smells in Java projects was investigated by Silva et al. (2024), who emphasized the importance of prompt engineering and in-context learning in enhancing detection accuracy. A generic prompt and a precise prompt that specifically mentioned the target smells–such as Blob, Data Class, Feature Envy, and Long Method–were both used in the study. The F-measure improvement for individual odors increased from 0.11 to 0.59, indicating that specific suggestions performed substantially better than generic ones. Furthermore, ChatGPT performed better at identifying critical-severity code smells than minor ones. This study demonstrates the possibility of in-context

learning to improve the adaptability of LLMs in software and emphasizes the significance of prompt specificity when utilizing large language models for software quality assessments.

Baumgartner et al. (2024) suggest an AI-driven pipeline that uses in-context learning to improve Large Language Models (LLMs) such as ChatGPT for restructuring data clumps in Git repositories. Their strategy incorporates human-in-the-loop techniques to improve refactorings proposed by AI and guarantee regulatory compliance with guidelines such as the EU-AI Act. The LLM is able to produce semantically relevant class names and the best refactoring recommendations by using in-context learning to supply it with precise examples and contextual data. Although issues like non-compilable code and resource limitations still exist, experimental findings showed that the pipeline could enhance code quality and maintainability by automating the detection and reworking of data clumps. This study emphasizes the importance of using LLMs to intricate software engineering processes while integrating human oversight and in-context learning.

An empirical study by Zhang and Saber (2024) evaluated how well Large Language Models (LLMs), particularly GPT-3.5 and GPT-4, detect code clones. The study used two datasets: BigCloneBench (human-generated clones) and GPTCloneBench (LLM-generated clones). Moreover, the study concentrated on several clone kinds, such as syntactic clones (Type-1 and Type-2) and semantic clones (Type-3 and Type-4). With recall rates of above 85% for Type-3 clones, their results show that GPT-4 consistently performed better than GPT -3.5 for all clone categories, especially for semantically complicated clones. The study demonstrated that GPT-4 outperformed GPT-3.5 in its comprehension of semantic subtleties in code. Furthermore, compared to human-generated clones, both models performed better on LLM-generated clones, indicating a bias toward identifying patterns in training data. The study emphasizes how crucial it is to adjust and prompt engineering in enhancing LLM capabilities for code clone detection.

3.3 Gap analysis

The conducted review reveals several gaps and contributions that this paper aims to address.

Firstly, there is a noticeable lack of studies exploring the use of ICL for code smell detection as compared to Fine-tuning or other approaches. Most existing studies fail to adopt a systematic approach; they do not comprehensively experiment with varying numbers of shots or leverage different LLMs to assess performance. Also, there is a gap in exploring the technique in more smells and programming languages. This absence of systematic experimentation leaves open questions about the optimal configurations and parameters for ICL in this context.

Secondly, and most critically, no study has directly compared fine-tuning and ICL for code smell detection. Such a comparison is crucial to determine which technique is more effective under specific conditions and to understand their respective trade-offs. Without this comparative analysis, it is challenging to establish a clear preference for either technique in practical applications.

To provide a clearer understanding of the contributions of this study, readers are encouraged to refer to the introduction of the paper and the research questions outlined in Table 1.

4 Methodology

Our followed methodology can be shown in Fig. 1. The details for each step are given in the following subsections.

4.1 Planning

4.1.1 Context selection

The context of this experiment is defined as an online setting involving real-life projects, as the datasets are collected from actual Java open-source projects. The experiment is conducted with professionals, ensuring relevance and practical applicability. Additionally, the experiment is specific in scope, focusing primarily on Java code with the addition of a small multilingual dataset.

4.1.2 Hypothesis

Based on the defined scope and objective, three aspects are measured: efficiency in terms of training time, efficiency in terms of the number of training samples required, and effectiveness measured by the F1 score. The experiment involves one factor, the training method, with two treatments: 1) in-context learning and 2) fine-tuning. Accordingly, we formulated three null hypotheses, each with two alternative hypotheses, as defined in Table 4. Furthermore, after testing these hypotheses, we developed research questions to guide our discussion. These research questions are directly related to the hypotheses and are provided in Table 1.

pc

4.1.3 Variable selection

To set up the experiment, we identified the dependent variables and the independent variables. Since our objective involves measuring two aspects–efficiency (training time and

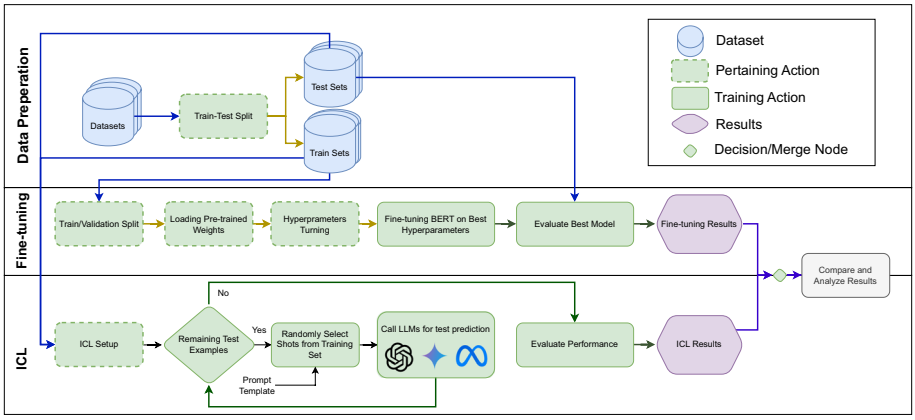

Fig. 1 The followed methodology showing ICL and fine-tuning pipelines

Table 4 Null hypothesis with corresponding alternatives

Null	Alternative
$\boldsymbol{H_0^1}$: There is no difference in effectiveness, measured by the F1 score, between in-context learning and fine-tuning for code smell detection.	$\boldsymbol{H_a^1}$: The performance, measured by the F1 score, for code smell detection using fine-tuning is higher than using in-context learning.
	$\boldsymbol{H_b^1}$: The performance, measured by the F1 score, for code smell detection using in-context learning is higher than using fine-tuning.
$\boldsymbol{H_0^2}$: There is no difference in efficiency, measured by training time in seconds, between in-context learning and fine-tuning for code smell detection.	$\boldsymbol{H_a^2}$: The time taken to train, measured in seconds, for code smell detection using fine-tuning is higher than using in-context learning.
	$\boldsymbol{H_b^2}$: The time taken to train, measured in seconds, for code smell detection using in-context learning is higher than using fine-tuning.
$\boldsymbol{H_0^3}$: There is no difference in efficiency, measured by the number of training samples required, between in-context learning and fine-tuning for code smell detection.	$\boldsymbol{H_a^3}$: The number of training samples required for fine-tuning is higher than for in-context learning.
	$\boldsymbol{H_b^3}$: The number of training samples required for in-context learning is higher than for fine-tuning.

training samples) and effectiveness (in terms of the F1 score)–these two serve as the dependent variables, with their changes being observed during the experiment. The treatment is defined as the technique used, whether it is ICL or fine-tuning.

To minimize the impact of confounding factors, we identified several variables to fix during the experiment:

1. **Evaluation Split**: Both techniques were tested on the same data split to ensure that differences in the dependent variables were attributable solely to the technique used.
2. **Prompt Used**: The prompt was fixed across all LLMs in the in-context learning tasks to enable a fair comparison.
3. **Model Parameters**: For in-context learning tasks, parameters such as `temperature`, `max_tokens`, `top_p`, `frequency_penalty`, and `presence_penalty` were fixed. Similarly, parameters like `learning rate` and saving the best model were fixed for fine-tuning tasks to ensure replicability and facilitate a fair comparison.

4.1.4 Subject Selection

This experiment does not involve any human subjects as it is entirely technology-based. The experiments are conducted using APIs provided by OpenAI for GPT and Google for Gemini. Additionally, the Kaggle environment with 2x GPU NVIDIA T4 (15 GiB VRAM) is utilized for fine-tuning tasks and for loading a quantized version of CodeLlama.

As **objects** of the experiment, eight large language models (LLMs) were selected–five for ICL and five for fine-tuning. The models selected for ICL are OpenAI GPT4o-mini, Google Gemini-Flash, and Meta's CodeLlama70B, GPT-2, and CodeGPT. For fine-tuning, the selected models are CodeBERT, GraphCodeBERT, and UniXcoder, GPT2, and CodeGPT. The two compact models, CodeGPT and GPT-2, were applied to both techniques, as they are small enough to be fully fine-tuned on our hardware and also being capable of in-context inference. They allow a controlled comparison in which the model is fixed and only the learning technique is the changed factor. Detailed descriptions of these models are provided in the data description section.

The rationale behind selecting these LLMs is their widespread use in the field Zhang et al. (2023). Furthermore, our goal was to ensure diversity among the selected models based on several criteria:

1. **Model Size**: Number of parameters in the model.
2. **Model Type**: Whether the model is open-source or proprietary.
3. **Training Data**: Whether the model is predominantly trained on code data or general data.

4.1.5 Design type

As this experiment is technology-based, it eliminates risks associated with ordering or participant experience. The experiment utilizes three datasets, which are detailed later in subsection 4.4 Data Description. The assignment of these datasets to each LLM for the ICL task is shown in Table 5, while the assignments for the fine-tuning task are presented in Table 6. In this assignment, we ensured that every LLM was evaluated on every dataset. This comprehensive testing strategy allows us to obtain results for each dataset across multiple LLMs, thereby strengthening the statistical validity and generalizability of our findings. Moreover, this approach mitigates potential biases arising from any specific model or dataset. The planned order of execution for these experiments is presented in Tables 5 and 6. The rationale for selecting some different models for fine-tuning compared to in-context learning is twofold. First, high-capacity models commonly used for ICL tasks are either closed-source and cannot be fine-tuned, such as Gemini, or so large that fine-tuning becomes impractical. Second, for fine-tuning, we selected commonly used models in the field to ensure relevance and feasibility. In addition, two compact models (CodeGPT and GPT-2) were applied to both techniques. Because these models are small enough to be fully fine-tuned on our hardware and also being capable of in-context inference. They allow a controlled comparison in which the model is fixed and only the learning technique is the changed factor.

4.1.6 Instruments

For our experiment, some instruments might involve the tools and packages used for training. We first used Kaggle for fine-tuning. Some Python packages like scikit-learn, pandas, and transformer are also used, which are mentioned later when used. As we planned for the experiment, we prepared all of these packages to be loaded for the environment.

Table 5 Experiment design for ICL

Model/Dataset	CoRT	MLCQ	Made Dataset
GPT4o-mini	X		
Gemini-flash	X		
CodeLlAMA70B	X		
CodeGPT	X		
GPT-2	X		
GPT4o-mini		X	
Gemini-flash		X	
CodeLlAMA70B		X	
CodeGPT		X	
GPT-2		X	
GPT4o-mini			X
Gemini-flash			X
CodeLlAMA70B			X
CodeGPT			X
GPT-2			X

Table 6 Experiment design for finetuning

Model/Dataset	CoRT	MLCQ	Made Dataset
CodeBERT	X		
GraphCodeBERT	X		
Unixcoder	X		
CodeGPT	X		
GPT-2	X		
CodeBERT		X	
GraphCodeBERT		X	
Unixcoder		X	
CodeGPT		X	
GPT-2		X	
CodeBERT			X
GraphCodeBERT			X
Unixcoder			X
CodeGPT			X
GPT-2			X

4.2 Selecting dataset

As part of establishing the context for this study, it was essential to select representative datasets. A key consideration in dataset selection was the availability of actual code samples and the quality of the data. We selected the CoRT dataset (Alazba et al. 2024) and the MLCQ dataset (Madeyski and Lewowski 2020). Additionally, we constructed an artificial dataset generated using GPT. The details of all selected datasets and the process for creating the artificial dataset are provided in subsection 4.4 Data Description.

An important consideration is that fine-tuning results from previous studies on these datasets have been reported. However, these studies used different test sets and reported varying performance metrics. To maintain greater control over the independent variables in our study and ensure the results are attributable solely to the treatment, we decided to

perform fine-tuning on the same test set used for ICL. Nonetheless, it is valuable to understand what others have achieved using these datasets, so we briefly report their results in the subsection 4.4 Data Description.

4.3 Training planning

In this section, we outline the training-specific planning details to facilitate the execution of the experiment and enhance its reproducibility. The planning includes details for both ICL and fine-tuning. Figure 1 illustrates the methodology we followed to evaluate the two techniques on the selected datasets.

4.3.1 In-context learning

Setup ICL is used for code smell detection. The technique is be experimented with different aspects: are **shots *(examples)***, models, and datasets. The planned setup for these variables is detailed in this section. To apply ICL effectively, several aspects must be considered when selecting examples, including the *number of examples and example formatting*.

Number of examples There is no analytical method to determine the optimal number of shots required for the best performance in every task. The number of examples provided to the model should be chosen based on factors such as task complexity, model capacity, context window size, example quality, and computational cost. A detailed description of these factors is presented in Table 7. However, despite following these general guidelines, LLM behavior remains unpredictable and nondeterministic (Ouyang et al. 2024). Arbitrarily selecting a number of shots x, y, z can lead to misleading conclusions regarding LLM performance.

In our experiment, we start with zero shots and incrementally increase the number of shots to observe their effect on the performance of the LLMs. While there is no analytical basis for preferring an odd or even number of shots, we aim to assess the impact of increasing the number of examples on LLM performance in code smell detection. Therefore, the specific parity (odd or even) of the examples is irrelevant as long as the number increases. For this reason, we decided to evaluate ICL using `0-shots, 1-shot, 3-shots, and 5-shots`.

Choosing examples Examples used in ICL should consist of {`Instruction, Output`} pairs for the task at hand. According to Min et al. (2022), ICL can achieve reasonable per-

Table 7 Factors influencing optimal number of few-shot examples

Factor	Description
Task Complexity	Complex tasks may require more examples.
Model Capacity	Large models can handle large number of examples but small models can lose context
Context Window	Total size of the instruction shouldn't overcome certain threshold of number of tokens set by the model provider
Example Quality	High quality representative example may be enough even with small numbers
Computational Costs	Large number of examples may require more computational cost.

formance even when random outputs are provided for the given instructions. This implies that even without prior ground truth values, ICL can still enable the LLM to generate correct answers for the given questions.

However, in our experiment, we do not intend to use this approach, as we already possess a sufficient amount of previously collected training data with ground truth labels. Instead, we randomly select different shots from the available training data. The samples are collected randomly because the ICL framework was originally proposed to provide random examples (Brown 2020). Future research could explore and evaluate alternative strategies for selecting these samples. The pseudo-code for the algorithm used to generate a few-shots from the current dataset is provided in Algorithm 1.

Algorithm 1 Shots Generator for Few-shot Examples

```
Data: Dataset and the number of shots needed (shots_n)
Result: String containing all the few-shot examples
Function GenerateShots(shots_n):
    Initialize: shots ← an empty string;
    for i = 0 to shots_n do
        Select a random index from the dataset;
        Retrieve the "Code" value from the dataset at the selected index;
        Retrieve the "Smelly" label (Smelly/No) at the selected index;
        Create a formatted example using the code and label;
        Add prompt text + label to the shots string;
    return shots;
```

Examples format LLMs are often inclined to produce verbose responses resembling human communication. However, our experiment requires the models to respond with only a single word or a short phrase to facilitate automated evaluation. To ensure consistency, we designed the examples to produce outputs aligned with the required format for each dataset. The formats of the prompts used in this experiment are detailed in the Appendix Section 10.1. Additionally, we employed randomization to mitigate the bias introduced by ordering selected examples.

Experiment setup In our experiment, we adopt a **Within-Subjects** design, where all the LLMs are evaluated with varying numbers of shots across all datasets. The models considered in our study are described in subsection 4.4 Data Description. Among the five ICL models, GPT4o-mini and Gemini are general-purpose closed-source models, CodeLlama-70B is an open-source model further pre-trained on coding-related tasks, and CodeGPT and GPT-2 are compact (approximately 124M-parameter) open-source models which are code-oriented and general-purpose, respectively. The models therefore span a wide range of capacities and training focuses. A full comparison of these five ICL models is presented in Table 19, with additional details provided in subsection 4.4 Data Description.

For the experiment, we run the three models using different numbers of few-shots for each dataset, as illustrated in Fig. 2. After selecting examples from the training split of the dataset, the test split is used to obtain results from the LLMs, following the process described in the pseudo-code in Algorithm 2. A quick comparison between the ICL models is present in Table 8.

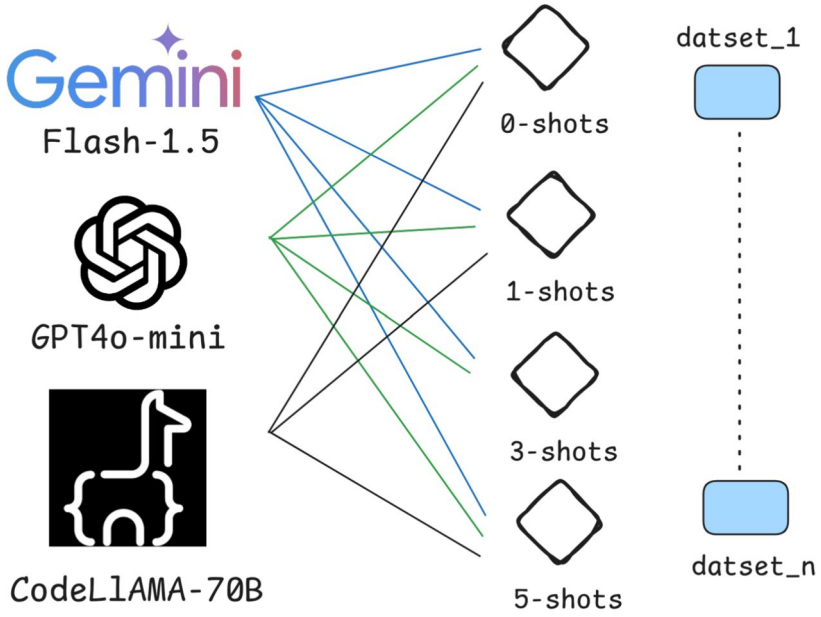

Fig. 2 Experiment workflow

Algorithm 2 File Processing and Response Generation

```
Data: Test dataset and model configuration
Result: Updated CSV file with model responses
Function ProcessFile(dataset, model):
    Load the CSV file into a table (dataframe);
    Create an empty list to store the model's responses;
    for each row in the table do
        Retrieve the "Code" value from the current row;
        Create a prompt using the 'shots' and the row's code;
        Send the prompt to the model to generate a response;
        Add the generated response (stripped of spaces) to the responses list;
        Wait for 4 seconds between requests to avoid overloading the system;
    Insert the responses into a new column called "Responses";
    Create a new file name based on the test file name;
    Save the updated table to the new file;
```

Hyper-parameters Setup To minimize the impact of confounding factors, hyper-parameters were standardized to consistent values across all experiments. Non-determinism in LLMs remains an open area of research, meaning that even with identical experimental setups and inputs, LLMs can produce varying outputs without traceable patterns (Ouyang et al. 2024). For instance, research such as Renze and Guven (2024) suggests that setting the `Temperature` hyper-parameter to any value has minimal impact on the performance of models like ChatGPT.

Table 8 Comparison of the five ICL models: Gemini, GPT-4o-mini, Code Llama-70B-Instruct, CodeGPT, and GPT-2

Feature	Gemini	GPT-4o-mini	Code Llama	CodeGPT	GPT-2
Model Type	Proprietary	Proprietary	Open-source	Open-source	Open-source
Focus	General	General	Programming	Programming	General
Open Source	No	No	Yes	Yes	Yes
Specialization	Conversational AI	Conversational AI	Code generation	Code generation	Text generation
Parameters	Undisclosed	Undisclosed	70B	∼124M	∼ 124M
API Availability	Yes	Yes	Local	Local	Local
Quantization	N/A	N/A	Yes (4-bit)	Not required	Not required

To ensure reproducibility, we fixed the hyperparameters for all LLMs. One particularly important parameter is the `temperature`, which controls the creativity or randomness in the model's responses. The effect of this parameter becomes more evident when the model generates longer outputs. However, in classification tasks like ours, where the model produces only a single word as output, there are insufficient studies that specify an optimal temperature value. To determine an appropriate value, we conducted an initial tuning experiment on one dataset, testing multiple values: 0, 0.3, 0.6, 1, and 1.5. The results of this experiment are shown in Table 9. As observed, there was no consistent pattern across temperature settings. For example, in the zero-shot setup, performance improved as the temperature decreased to 0, but then declined again between 0.6 and 0.3. Moreover, no single value consistently outperformed others across all configurations. Therefore, to avoid any setup bias, we selected the default value provided by OpenAI, which lies approximately in the middle of the typical range (0–2). Moreover, in the table, temperature of 1 was the best across two settings, which is better than other temperatures, and it was the second best, showing good performance in the zero-shot setting. All other hyperparameters were also set to their OpenAI defaults. Four hyper-parameters were selected for control during our experiment: `Temperature`, `max_tokens`, `top_p`, `frequency_penalty`, and `presence_penalty`, as described in Table 10.

Table 9 Performance across temperatures and shot counts

Temp / # shots	0	1	3	5
0.0	0.58	**0.69**	0.72	**0.73**
0.3	0.58	0.68	0.73	**0.73**
0.6	**0.60**	0.66	0.73	0.72
1.0	0.59	0.65	**0.75**	**0.73**
1.5	0.57	0.65	0.74	0.70

The `max_tokens` parameter does not have a default value specified in the OpenAI API documentation[1]. Since our experiment requires responses consisting of only 1-2 words, we set `max_tokens` to 10. This limit reduces the likelihood of additional text being introduced by the language model at the beginning of the response.

Models API Setup Closed LLMs such as: `Gemeni-flash-1.5` and `GPT4o-mini` use some API to be able to interact with the models. Setting hyper-parameters, as discussed in Line 12, and getting a response using the API may be different depending on the vendor. For Gemini, we have to set some safety blocking parameters in order to prevent any wrong blocking for our requests. OpenAI does not provide a similar structure, but we mitigate any blocking during execution by repeating the request or using another session during the experiment.

4.3.2 Fine-tuning

In fine-tuning, we utilized the training set to adjust the pre-trained model weights for detecting code smells. The detailed plan for the fine-tuning process is outlined below.

Training environment For fine-tuning, we use the Kaggle environment, which provides two NVIDIA T4 GPUs, each with 15 GiB of VRAM, for a total of 30 GiB, along with 30 GiB of system RAM. This setup ensures sufficient computational resources for the task.

Dataset split While the test split has been fixed across both in-context learning (ICL) and fine-tuning experiments, fine-tuning requires an additional validation set for hyperparameter tuning and model selection. This ensures that the best model is evaluated on the original test set, thereby avoiding any potential data leakage.

Table 10 Hyper-parameters, their descriptions, and the explanations of their set values

Hyper-parameter	Description	Value
`temperature`	Controls the randomness of the output. Higher values produce more diverse results, while lower values make outputs more deterministic.	1
`max_tokens`	Limits the number of tokens in the output, defining the maximum response length.	10
`top_p`	Enables nucleus sampling, controlling how much of the probability distribution is considered for token selection.	1
`frequency_penalty`	Reduces the model's tendency to repeat tokens by penalizing repeated words or phrases.	0
`presence_penalty`	Discourages tokens that have already appeared in the context, reducing redundancy.	0

[1] https://openai.com/index/openai-api/

To create the validation set, we applied an 80:20 split to the training set. This approach is supported by the findings of Alazba et al. (2023), who identified the 80:20 split ratio as the most commonly used in code smell detection tasks.

Training package For fine-tuning, we use the Huggingface Trainer package, which offers a user-friendly API for fine-tuning. It allows the specification of training arguments and includes built-in callbacks, such as saving the best model during training.

Data collected To achieve the goals of this experiment, we measure several performance metrics. The primary metric for this study is the F1 score. Additionally, we report Recall, Precision, and Matthews Correlation Coefficient (MCC) to provide a more comprehensive understanding of model performance, as these metrics are widely used in the literature. We also record the training time for each model on each dataset.

Fixed training arguments For consistency across all models, we fixed specific settings for training:

1. The number of training epochs is fixed at 10. To avoid overfitting, we use a callback to save the best model based on the highest validation F1 score, and this model is used for the final evaluation.
2. The learning rate is set to `5e-5`, following the recommendation for fine-tuning BERT models (Kenton and Toutanova 2019), and it was decreased automatically during training to avoid overfitting.
3. The batch size is fixed at 16 due to capacity limitations.
4. The Adam (Diederik 2014) optimizer is used, as it is widely adopted in deep learning systems.

Figure 3 illustrates the model architecture, which consists of a pre-trained transformer model with an attached classification head tailored to the number of classes in each dataset. Note that we experimented with five different models.

Results without training Another aspect we aim to evaluate is the performance of the model without training. By attaching a classification head, we evaluate the model directly on the test set. Prior to recording this, we hypothesize that the results are random, as the model has no prior fine-tuning and thus predicts each class with equal probability. For example, in the CoRT Feature Envy dataset, the accuracy is expected to be $1/\text{number of classes} = 0.5$. However, we record this empirically to confirm our hypothesis.

The rationale for recording this is to enable a comparison with the zero-shot setting in ICL. This simulates a scenario without labeled data for fine-tuning a model. Although this approach is not typically used in fine-tuning, it provides valuable insight into whether the zero-shot performance in ICL arises from the model's inherent capabilities or is simply random.

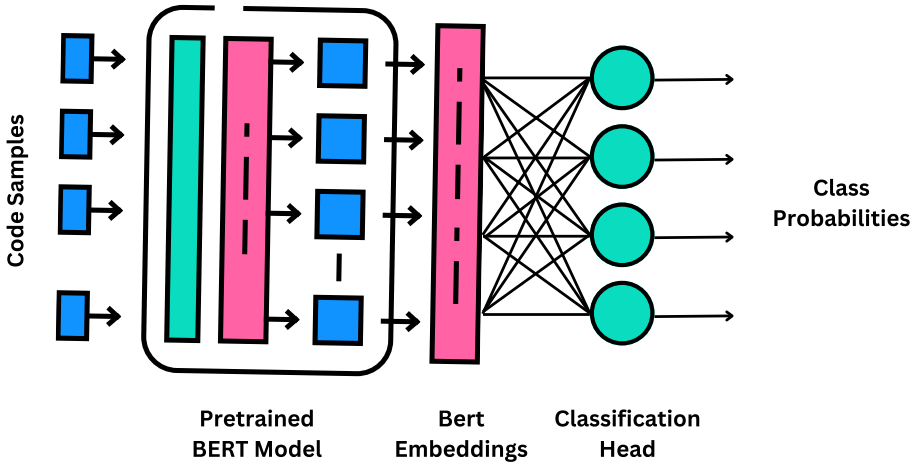

Fig. 3 Model architecture

4.4 Datasets

This subsection provides details about the selected datasets, including their domain, size, the code smells they address, and results from previous studies using these datasets. Table 11 summarizes the datasets used in our study, along with the size of the subsets employed in our experiments.

4.4.1 CoRT dataset

The CoRT dataset was developed by Alazba et al. (2024). It is a multi-class dataset collected from 103 diverse Java open-source projects, using a set of diversity criteria designed to ensure the dataset's representativeness and generalizability. The dataset annotations were derived from prior studies cited in the paper (Alazba et al. 2024).

Table 11 Overview of Datasets Showing Size, Number of Smells, and Samples

Dataset	Training Size	Test Size	Smells	Languages
CoRT - FE	1200	300	Feature Envy	Java
CoRT - LM	1200	300	Long Method	Java
CoRT - GC	1200	300	God Class	Java
CoRT - DC	1200	300	Data Class	Java
MLCQ	1077	269	Blob, Data Class, Feature Envy, Long Method	Java
Artificially Made	384	96	Code Clone, Data Class, Feature Envy, God Class, Long Method, Long Pram List.	Jave, Python, C++, JavaScript

The dataset addresses four code smells. Two are class-level smells: God Class and Data Class. The other two are method-level smells: Long Method and Feature Envy. The dataset is organized into four subsets, one for each smell, where each file contains a code sample along with a binary label (1 or 0) indicating whether the sample exhibits the corresponding smell. The dataset comprises approximately 107K samples in total.

In the same paper, they tested the dataset on a transformer model they developed, reporting F1 scores of 94.48% for God Class, 95.89% for Data Class, 88.68% for Feature Envy, and 88.87% for Long Method. However, due to the dataset's large size, we used a subset for our experiments. Specifically, we selected 1200 samples from each smell subset for training and 300 samples from each smell subset for testing.

4.4.2 MLCQ dataset

About the dataset The MLCQ dataset (Madeyski and Lewowski 2020) was developed to address limitations in earlier code smell datasets, which often included low-quality data from outdated projects and lacked input from experienced professionals. To overcome these issues, the MLCQ dataset was created in collaboration with industry professionals, drawing code samples from active and relevant open-source projects on GitHub. Its goal was to improve the quality of code smell research and introduce auxiliary data, such as developer experience and background, for more nuanced analysis (Madeyski and Lewowski 2023). The dataset comprises 14,739 reviews of 4,770 unique code samples from 523 projects. Key code smells include Blob, Data Class, Feature Envy, and Long Method, which were selected for their prominence in software engineering literature. The dataset's reviewer diversity and four-tier severity scale provide insights into how developer experience influences code smell perception, aiding the development of future automated detection tools (Madeyski and Lewowski 2023).

Use cases Several studies have utilized the MLCQ dataset to evaluate machine learning algorithms for detecting code smells. One such study (Madeyski and Lewowski 2023) highlighted the following results:

Best-Performing Algorithms: Random Forest and Flexible Discriminant Analysis (FDA) emerged as the most effective classifiers for code smell detection, with Random Forest achieving notable success across multiple code smells.

Detection Accuracy by Code Smell: For Long Method, Random Forest achieved the highest performance with a median MCC of 0.81. For Blob, Random Forest reached a median MCC of 0.51. FDA demonstrated the best performance for Feature Envy, with a median MCC of 0.31, while Random Forest achieved a median MCC of 0.57 for Data Class detection.

General Findings: Long Method detection yielded the most consistent results, while Feature Envy detection was more challenging due to data imbalances. Matthews Correlation Coefficient (MCC) was used as the primary performance metric due to its suitability for imbalanced datasets.

Another study (Škipina et al. 2024) evaluated various machine learning models on the MLCQ dataset, focusing on Feature Envy and Data Class detection. For Feature Envy, Ran-

dom Forest achieved the highest F1-score of 0.85, followed by Neural Networks with an F1-score of 0.80 and Support Vector Machine (SVM) with an F1-score of 0.78. For Data Class, Random Forest achieved an F1-score of 0.82, Neural Networks an F1-score of 0.81, and SVM an F1-score of 0.79.

These results indicate that Random Forest consistently outperformed other models in detecting Feature Envy and Data Class code smells within the MLCQ dataset. The findings underscore the importance of algorithm selection and metric optimization to improve detection accuracy and reliability.

4.4.3 Artificially constructed dataset

Due to the lack of publicly available datasets containing actual code samples suitable for ICL and fine-tuning LLMs, we constructed an artificial dataset generated using GPT. This dataset was designed to be multilingual, covering multiple programming languages and domains.

The dataset generation process involved prompting GPT to produce code samples containing specific code smells from various domains. The targeted code smells included Code Clone, Data Class, Feature Envy, God Class, Long Method, and Long Parameter List. The programming languages considered were Python, Java, JavaScript, and C++.

It is important to clarify the scope of code clone detection in this dataset. Traditional code clone detection tools operate at the project level, scanning entire codebases to identify duplicate fragments across files. However, in this study, the task is formulated as a *classification* problem: given a self-contained code sample, the model must determine whether it exhibits a code clone smell. This is analogous to how other smells are treated in the dataset; for instance, a God Class sample contains a single class, and the model classifies whether it exhibits the smell, without requiring the full project context.

Consequently, the code clone samples in the dataset are structured in two forms: (1) a single class containing multiple methods with duplicated logic, or (2) two short classes provided together in the same sample that share common duplicated segments. Examples of both forms are shown in subsection 10.2 and 10.2 in the Appendix. In a practical setting, project-level clone detection would require a separate engineering layer (e.g., a tool that extracts candidate code pairs from a codebase) that feeds the extracted samples into the classification model. The model's responsibility is limited to classifying a given code sample, while the broader detection pipeline falls outside the scope of this study.

For each domain, smell, and language combination, a prompt was sent to GPT to generate a code to exhibit the smell in this language from this domain. The prompt used is available in the Appendix. The results were then collected, and the dataset was constructed. For each domain, smell, and language combination, a prompt was sent to GPT to generate a code to exhibit the smell in this language from this domain. The results were then collected, and the dataset was constructed.

The resulting dataset comprises 384 training samples and 96 test samples, providing a balanced and diverse representation of the targeted code smells across the selected programming languages and domains. The dataset can be found in the GitHub Repository[2].

[2] https://github.com/nawafalomari/ics500/tree/main/Datasets/GPT%20Dataset

4.4.4 Dataset validation

Since the dataset was constructed by prompting ChatGPT, it is crucial to assess its validity, as some samples may have been generated with apparent code smells but could, in reality, be incorrect. To ensure reliability, we conducted a validation study following a statistically grounded sampling approach. Using Cochran's formula with a 95% confidence level and a 5% margin of error, the required sample size for a population of 480 samples was calculated to be 214. We randomly selected 215 samples from the dataset for validation. Since the original dataset is balanced, the random selection resulted in class frequencies range from 29 to 44 instances, yielding an imbalance ratio of 1.52. The normalized entropy of the class distribution is 0.995, indicating a near-uniform distribution with no severely underrepresented class in the manual validation set.

Two independent reviewers participated in the validation process. Both reviewers are graduate students with experience in software engineering and code smells. Each reviewer independently assessed whether each sample correctly exhibits the intended code smell by labeling it as "Valid" or "Invalid." The reviewers worked independently without knowledge of each other's assessments. After independent review, a third reviewer resolved all disagreement cases.

The inter-rater agreement between the two reviewers, measured using Cohen's Kappa Cohen (1960), was $\kappa = 0.51$, indicating moderate agreement. The raw agreement rate was 93.0% (200 out of 215 samples). The moderate Kappa value, despite the high raw agreement, is a well-documented phenomenon known as the Kappa paradox (Feinstein and Cicchetti 1990). When one category is highly prevalent, as in our case where the vast majority of samples are valid (prevalence index = 0.85), the expected agreement by chance becomes very high, which deflates the Kappa statistic. To account for this, we also report the Prevalence-Adjusted Bias-Adjusted Kappa (PABAK) (Byrt et al. 1993), which corrects for the effect of prevalence and bias on Kappa. The PABAK was 0.86, indicating almost perfect agreement, which more accurately reflects the actual level of concordance between the two reviewers. After conflict resolution, **205 out of 215 samples (95.3%)** were deemed valid. The 10 invalid samples were mainly borderline cases rather than critical errors. Most of them involved the Data Class smell, where the generated class contained minor computed properties or small utility functions that could be interpreted as exceeding the typical definition of a data class, making the boundary between a valid data class smell and a class with minimal behavior ambiguous. No samples were found to contain entirely incorrect or unrelated code smells. This high validity rate confirms that the artificially constructed dataset is reliable for the purposes of this study. Table 12 presents examples of different types of smells drawn from the dataset.

4.4.5 Comparative dataset characteristics

To further assess the characteristics of the artificially constructed dataset, we compare key code metrics across all three datasets. Since the CoRT dataset was preprocessed with newline characters and comments removed, we report the number of Java statements (approximated by semicolon count) as a proxy for code size. For MLCQ and the GPT Dataset, where newlines are preserved, we report lines of code (LOC). Additionally, we report token count (whitespace-delimited), an approximation of cyclomatic complexity (CC) based on

Table 12 Samples from the validation in the Artificially Generated dataset

Smell	Code Sample
Code Clone	`class Movie { constructor(title, director, releaseYear) { this.title = title; this.director = director; this.releaseYear = releaseYear; } getDetails() { return '${this.title}, directed by ${this.director}, released in ${this.releaseYear}'; } } class TVShow { constructor(title, creator, seasons) { this.title = title; this.creator = creator; this.seasons = seasons; } getDetails() { return '${this.title}, created by ${this.creator}, with ${this.seasons} seasons'; } }`
Feature Envy	`def calculate_discounted_price(self, customer): if customer.loyalty_points > 100: discount = 0.2 elif customer.loyalty_points > 50: discount = 0.1 else: discount = 0 return self.price * (1 - discount)`
Long Parameter List	`def calculate_production_cost(material_cost, labor_cost, overhead_cost, shipping_cost, taxes, discounts, markup_percentage, production_time, machine_hours, maintenance_cost, waste_percentage, environmental_fees): total_cost = (material_cost + labor_cost + overhead_cost + shipping_cost + taxes - discounts) * (1 + markup_percentage / 100) per_hour_cost = total_cost / production_time adjusted_cost = per_hour_cost + (maintenance_cost / machine_hours) + (total_cost * waste_percentage / 100) + environmental_fees return adjusted_cost }`

decision keyword counts, the number of method definitions detected via language-aware pattern matching, and vocabulary size (the number of unique tokens per sample). For the GPT Dataset, which spans Python, Java, JavaScript, and C++, the method detection patterns account for multiple language conventions (e.g., `def` for Python, access modifiers for Java, `function` for JavaScript). The results are shown in Table 13.

The three datasets exhibit different characteristics, which is expected given their distinct origins and purposes. The CoRT and MLCQ datasets, collected from real open-source projects, contain code samples that vary widely in size and complexity. For example, the large gap between the mean and median values in CoRT God Class (mean 151.8, median 85.0 statements) and CoRT Data Class (mean 173.7, median 25.0 statements) indicates a skewed distribution with some very large samples. The GPT Dataset, by contrast, shows more consistent and focused samples (mean 24.3, median 22.0 LOC), which reflects its design goal of clearly isolating specific code smells without the noise and boilerplate that accompany production code.

This difference in scale is natural and does not undermine the dataset's utility. The method-level smells in the real-world datasets (CoRT Feature Envy and Long Method) are comparable in size to the GPT Dataset, with median values of 4.0 and 3.0 statements respectively. Furthermore, the GPT Dataset contributes diversity that the other datasets lack: it covers four programming languages (Python, Java, JavaScript, and C++) and includes code smells such as Code Clone and Long Parameter List that are not present in CoRT or MLCQ. Importantly, the experimental results obtained from the GPT Dataset followed similar performance patterns to those observed on the real-world datasets, confirming that the models' behavior generalizes across datasets of varying characteristics, which further supports the robustness of our findings.

4.5 LLMs

4.5.1 Gemini Flash

Gemini Flash is a state-of-the-art language model developed by Google DeepMind, designed to excel in natural language understanding and generation. As part of Google's Gemini series, this model integrates language and multimodal capabilities within a unified framework to enhance response accuracy, user engagement, and contextual comprehension. Gemini Flash prioritizes fast and lightweight performance, making it particularly well-suited for applications requiring real-time responses with minimal computational resources. This advancement represents a significant step toward improving resource efficiency in large-scale language models without compromising the quality of outputs (Team et al. 2023).

4.5.2 GPT-4oMini

GPT-4oMini is a compact variant of OpenAI's GPT-4 architecture, optimized for scenarios demanding high performance in resource-constrained environments. Unlike the full-scale GPT-4, which is extensively trained on a vast number of parameters, GPT-4oMini delivers comparable language understanding and generative capabilities with reduced computational requirements. This model is particularly advantageous for deployment on mobile and edge devices, enabling broader accessibility to high-quality natural language processing appli-

Table 13 Comparative characteristics of the three datasets

			Size		Tokens		CC		Methods		Vocabulary	
Dataset	Size Metric	N	Mean	Med	Mean	Med	Mean	Med	Mean	Med	Mean	Med
CoRT – God Class	Stmts	1500	151.8	85.0	2302.5	1203.5	39.0	12.0	29.9	14.0	165.3	123.0
CoRT – Data Class	Stmts	1500	173.7	25.0	2820.9	357.0	66.9	3.0	50.8	9.0	81.5	60.0
CoRT – Feature Envy	Stmts	1500	12.8	4.0	201.5	72.0	5.5	1.0	1.4	1.0	33.8	19.0
CoRT – Long Method	Stmts	1500	14.3	3.0	212.8	49.0	6.3	1.0	1.2	1.0	35.2	16.0
MLCQ	LOC	1367	202.9	74.0	741.9	262.0	29.6	11.0	16.0	5.0	159.4	92.0
GPT Dataset	LOC	480	24.3	22.0	85.6	68.0	3.7	1.0	3.7	1.0	32.5	25.0

cations in settings where hardware limitations might otherwise pose a challenge (Brown 2020).

4.5.3 CodeLlama-70B

CodeLlama-70B, part of Meta's LLaMA (Large Language Model Meta AI) family, is a specialized language model tailored for code-related tasks, including generation, comprehension, and completion. With a substantial 70 billion parameters, CodeLlama-70B is engineered to assist with a wide range of programming activities, such as debugging, code summarization, and managing complex coding challenges. Its capabilities significantly enhance developer workflows by providing intelligent code suggestions and improving the overall quality and coherence of generated code (Roziere et al. 2023).

4.5.4 CodeBERT

CodeBERT, developed collaboratively by Microsoft Research and Hugging Face, is a bidirectional language model pre-trained on a substantial corpus of source code from multiple programming languages. Built on BERT's architecture, CodeBERT is designed to understand both the syntax and semantics of code, enabling it to excel in tasks such as code completion, summarization, and retrieval. It is particularly suited for applications like code search engines and automatic documentation generation, bridging the gap between natural language processing and code representation learning (Feng et al. 2020).

4.5.5 GraphCodeBERT

GraphCodeBERT, an extension of CodeBERT developed by Microsoft Research and Hugging Face, integrates structural information from source code to enhance its semantic understanding. By leveraging data flow graphs and the intrinsic syntactic features of programming languages, GraphCodeBERT captures deeper semantic relationships within code. Pre-trained on a diverse corpus of source code, it demonstrates exceptional performance in tasks such as code translation, refinement, and defect detection. Its graph-aware design is particularly advantageous for applications requiring context-rich code analysis, such as dependency tracking and advanced code understanding (Guo et al. 2020).

4.5.6 UniXcoder

UniXcoder, another innovative model from Microsoft Research, is a unified cross-modal pre-trained model tailored for programming-related tasks. Supporting both natural language and code, UniXcoder enables seamless integration between the two modalities. It is pre-trained on a variety of tasks, including code generation, completion, summarization, and retrieval. With its unique architecture, UniXcoder concurrently processes diverse input modalities, such as textual descriptions and code. This versatility makes it highly effective for tasks like code search, automatic documentation generation, and cross-modal reasoning, providing a valuable tool for developers seeking to bridge the gap between human-readable documentation and executable code (Guo et al. 2022).

4.5.7 CodeGPT

CodeGPT (Lu et al. 2021) is a compact (approximately 124M-parameter) GPT-2-style autoregressive language model that has been further pre-trained on source code. It is designed for code understanding and generation tasks such as code completion and code generation. Because of its small size, CodeGPT can be fully fine-tuned on modest hardware while also being capable of text generation, which makes it suitable for applying both learning techniques to the same model.

4.5.8 GPT-2

GPT-2 (Radford et al. 2019) is a general-purpose autoregressive language model from OpenAI, pre-trained on a large corpus of English text. We use the base (approximately 124M-parameter) variant. Unlike CodeGPT, GPT-2 is not specialized for source code, which makes it a useful general-purpose counterpart of comparable size. Its compactness likewise allows it to be both fully fine-tuned and used for in-context inference, enabling a controlled comparison of the two techniques on an identical model.

5 Experiment and Evaluation

The experimental plan described earlier was executed in two main parts: focusing on ICL and fine-tuning. Each is detailed in the following sections.

5.1 Evaluation

To compare the two techniques, we used a set of metrics that are widely used in code smell detection tasks (Alazba et al. 2023). The metrics used are:

Precision Precision (Jones et al. 2000) indicates the ratio of correctly identified positive instances out of all instances classified as positive.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{1}$$

Recall Recall (Jones et al. 2000) represents the proportion of actual positive instances that were accurately identified by the model.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{2}$$

F1 Score The F1 Score (Jones et al. 2000) serves as the harmonic mean of Precision and Recall, aiming to balance both metrics.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

Matthews Correlation Coefficient (MCC) (Matthews 1975) The MCC is a comprehensive measure for binary classification that takes into account all four elements of the confusion matrix (True Positives, False Positives, True Negatives, and False Negatives), making it especially valuable for datasets with class imbalance.

$$\text{MCC} = \frac{(\text{TP} \times \text{TN}) - (\text{FP} \times \text{FN})}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \tag{4}$$

The MCC ranges between -1 and +1 :

- $+1$ implies a perfect prediction.
- 0 indicates no better performance than random prediction.
- -1 implies complete discordance between prediction and actual labels.

5.2 Validation

For the validation, we split each dataset with an 80:20 ratio as a train-test split since it is the most common split method used in this task (Alazba et al. 2023). For the fine-tuning, since it involves hyperparameter tuning, we took part of the training set as a validation set to keep the test set fixed between both techniques.

5.3 In-context learning

For ICL, the test set was used to generate predictions from each LLM, followed by an evaluation of their outputs. The subsequent sections elaborate on the execution process. The actual code can be found in a public GitHub Repository[3]

5.3.1 Environment setup

The environment for ICL was configured using Python scripts to interact with the OpenAI Completion API and Google Gemini API. API keys were obtained for both services, and clients were set up to query the models and retrieve their outputs.

For CodeLlama-70B-Instruct, which lacks a public API, the tool Ollama[4] was employed. Ollama enables local deployment of the model on a device and provides an API endpoint for inference.

Each call to the model was configured with the following parameters:

```
{
  "temperature": 1,
  "max_tokens": 10,
  "top_p": 1,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "system": "Only give one best answer without any explanation.
             The answer should be from the choices given only."
}
```

[3] https://github.com/nawafalomari/ics500/tree/main

[4] https://ollama.com/

These parameters ensure consistency across model outputs, focusing on obtaining a single, optimal response without additional explanations.

5.3.2 Loading the dataset

The datasets were loaded using the `Pandas`[5] library in Python. Each dataset was divided into a training split, which provided demonstration examples for the few-shot setting, and a test split, which was used to evaluate the model's performance.

5.3.3 Generating predictions

With the environment set up and datasets prepared, predictions were generated using a Python `.map()` function. This function iterates over all the samples in the test set. Each code sample was injected into a predefined prompt template corresponding to the specific dataset, as described in the in-context learning planning section, and the prompt templates can be shown in the Appendix section 10.1.

In the case of few-shot prompting, random samples from the training set were selected and injected into the same prompt template. These few-shot examples were concatenated with their respective target labels, providing the model with additional context for prediction. Finally, the completed prompt was sent to the LLM to obtain predictions. As discussed in the plan, we have run the models in different shot settings, including 0-shot, 1-shot, 3-shots, and 5-shots.

Moreover, the predictions were generated for each dataset from each of the three described models, once for each shot setting, totaling 120 execution variations.

Figure 4 illustrates an example of the feature envy prompt template. The template shows how the test sample is injected, with the target field left empty for the model to predict. If

Prompt Template

```
In terms of Feature Envy Smell, the following method can be classified
as Smelly if it contains Feature Envy or No if it does not contain
the smell. Only answer
with the word Smelly or the word No.
The code sample: {{INPUT}}
The best classified as: {{TARGET}}
```

Sample Injected

```
In terms of Feature Envy Smell, the following method can be classified
as Smelly if it contains Feature Envy or No if it does not contain
the smell.
Only answer with the word Smelly or the word No.
The code sample: public static void main(arg[]){..}...
The best classified as:
```

Fig. 4 Prompt example

[5] https://pandas.pydata.org/

few-shot examples are included, they are concatenated in the same manner, but with their targets explicitly provided to guide the model.

5.3.4 Results evaluation

Since we have the model predictions as well as the true labels for the test set, we used the scikit-learn[6] package to calculate the classification report, which contains the F1 Score, Precision, Recall, and accuracy. Also, we used another function to get the MCC. The results were finally recorded in an online Google sheet for later analysis.

5.4 Fine-tuning

The fine-tuning process is more complex than In-Context Learning (ICL), as it involves updating the model's weights to adapt it into a classifier tailored for the provided dataset. The following sections detail the steps undertaken during this process. The actual code can be found in a public GitHub Repository[7]

5.4.1 Environment setup

Fine-tuning requires significant computational resources, particularly GPUs with high memory capacity. To meet these requirements, the Kaggle environment was utilized, offering access to two GPUs, each with 30 GiB of VRAM. This setup is well-suited for fine-tuning large language models (LLMs) of a reasonable size.

The models were loaded individually during their respective training sessions using the Hugging Face `transformers`[8] library. Each model was configured with a classification head, an output layer designed for the specific classification task.

For multi-class classification problems, such as those in the MLCQ and the constructed dataset, the classification head employed a `softmax` activation function with the number of neurons equal to the number of classes. For binary classification tasks, such as in the CoRT dataset, the classification head used a `sigmoid` activation. Here are the equations for Softmax and Sigmoid:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

We used for binary classification in CoRT datasets a Binary cross entropy loss

The **Binary Cross-Entropy** is used for binary classification tasks and is defined as:

[6] https://scikit-learn.org/stable/

[7] https://github.com/nawafalomari/ics500/tree/main

[8] https://huggingface.co/docs/transformers/en/index

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where:

- N is the number of samples,
- y_i is the true label (0 or 1),
- $\hat{y}_i$ is the predicted probability for class 1.

While for the other two datasets for multi-class classification, we used the **Categorical Cross-Entropy** , which is usually used for multi-class classification tasks and is defined as:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{ij} \log(\hat{y}_{ij})$$

where:

- N is the number of samples,
- C is the number of classes,
- y_{ij} is a binary indicator (1 if sample i belongs to class j, otherwise 0),
- $\hat{y}_{ij}$ is the predicted probability for sample i being in class j.

5.4.2 Data preprocessing

The training and test splits were prepared for fine-tuning. A portion of the training set, comprising 20%, was set aside as the validation set, maintaining an 80:20 split. Labels were encoded as binary (0 and 1) for the CoRT dataset. For the multi-class datasets, one-hot encoding was applied to the labels.

The code samples were tokenized using the tokenizer associated with each model. The maximum token length was set to 512, with shorter samples padded to this length. Padding involved adding a special padding token to shorter samples, ensuring a uniform input length across all samples. These padding tokens are typically ignored by the model during training, so they do not influence the results.

5.4.3 Training

Each model was trained for 10 epochs using a learning rate of 5×10^{-5} and a batch size of 16. The model was evaluated on the validation set every 100 training steps. To mitigate overfitting, a "save best model" callback was employed. This approach saved the model's weights as a checkpoint whenever the validation F1 score improved compared to the previously saved weights. Training continued until the completion of all 10 epochs.

This strategy was chosen over the commonly used early stopping technique, which halts training if the validation loss increases over several epochs. While effective in many scenarios, early stopping can prematurely terminate training due to the presence of local minima in

the loss function. By continuing the training, better model weights may be discovered even if the validation loss fluctuates temporarily.

Each model from the described models was fine-tuned on each dataset alone; we ended up with 30 different fine-tuned models.

5.4.4 Results evaluations

Once the fine-tuning process was completed, the best model from each of the three selected LLMs was used to generate predictions on the test set. Using the true labels and the predicted labels, evaluation metrics were calculated with the `scikit-learn`[9] library. The metrics included F1 score, Precision, Recall, Accuracy, and Matthews Correlation Coefficient (MCC). Additionally, the training time was recorded to test $\boldsymbol{H_0^2}$.

6 Experiment results

This section presents the results obtained from the conducted experiments. First, the performance metrics for the models is outlined. Following this, statistical tests are performed to evaluate the hypothesis stated earlier. Finally, the results are analyzed and discussed to address the research questions.

6.1 Results

The results of the experiments are divided into two main sets: one for ICL and the other for fine-tuning. The detailed results for each are presented in the following subsections.

It is important to note the difference in how results are presented across datasets, which stems from the training approach adopted for each. For the CoRT dataset, we followed the approach used in the original study (Alazba et al. 2024), where each code smell is treated as an independent *binary* classification task (smelly vs. not smelly). Accordingly, a separate model is trained and evaluated for each smell, and the results are reported per smell. For MLCQ and the Artificially Constructed Dataset, we adopted a *multi-class* classification approach, where a single model is trained to distinguish among all smell categories simultaneously. Therefore, the results for these datasets are reported as a single entry per model, since each dataset has only one classifier rather than one per smell. This distinction in presentation is a direct consequence of the training approach used for each dataset.

6.1.1 In-context learning

For ICL, a total of 72 experimental variations were conducted, covering different combinations of models, datasets, and the number of shots. As we experiment with three LLMs (gpt4o-mini, gemini1.5-flash, and codellama-70B) with 4 shot settings (0, 1, 3, 5), where the number represents how many examples we are giving from the training pool, and 6 datasets (CoRT 4 subsets, MLCQ, and the Artificially Made). The results are presented in two tables for improved readability: Tables 14 and 15. Additionally, Fig. 5 compares the F1 score across various numbers of shots. As expected, the results generally improve with an

[9] https://scikit-learn.org/stable/

Table 14 ICL results for CoRT Feature Envy, Long Method and Data Class

LLM	Shots	CoRT – FE					CoRT – LM					CoRT – DC				
		F1	MCC	ACC	Pres	Reca	F1	MCC	ACC	Pres	Reca	F1	MCC	ACC	Pres	Reca
GPT4o-mini	0	0.79	0.57	0.79	0.79	0.79	0.75	0.55	0.76	0.76	0.75	0.55	0.33	0.61	0.75	0.61
	1	0.71	0.43	0.71	0.72	0.71	0.77	0.55	0.78	0.76	0.79	0.65	0.31	0.65	0.66	0.65
	3	0.70	0.43	0.71	0.73	0.71	0.80	0.61	0.82	0.80	0.82	0.66	0.35	0.67	0.69	0.67
	5	0.75	0.51	0.75	0.76	0.75	0.85	0.69	0.86	0.84	0.86	0.69	0.45	0.70	0.75	0.70
Gemini 1.5-flash	0	0.71	0.44	0.71	0.73	0.71	0.87	0.74	0.89	0.87	0.87	0.60	0.35	0.64	0.73	0.64
	1	0.66	0.27	0.62	0.65	0.62	0.81	0.62	0.83	0.81	0.82	0.69	0.41	0.69	0.72	0.69
	3	0.68	0.37	0.68	0.69	0.68	0.85	0.50	0.87	0.85	0.86	0.73	0.50	0.74	0.76	0.74
	5	0.73	0.47	0.73	0.74	0.73	0.83	0.66	0.85	0.83	0.84	0.72	0.48	0.73	0.76	0.73
CodeLlama-70B	0	0.48	0.08	0.54	0.55	0.54	0.51	0.07	0.54	0.56	0.52	0.43	-0.01	0.50	0.49	0.50
	1	0.49	0.03	0.55	0.56	0.55	0.52	0.02	0.55	0.57	0.55	0.50	0.14	0.55	0.59	0.55
	3	0.40	-0.08	0.48	0.44	0.48	0.37	-0.12	0.46	0.41	0.46	0.44	0.03	0.51	0.52	0.51
	5	0.46	0.01	0.50	0.52	0.51	0.53	0.03	0.55	0.57	0.55	0.44	0.06	0.52	0.55	0.52
CodeGPT	0	0.34	-0.11	0.45	0.39	0.46	0.37	-0.23	0.50	0.36	0.39	0.35	-0.03	0.49	0.45	0.49
	1	0.39	-0.02	0.48	0.48	0.49	0.46	0.03	0.65	0.52	0.51	0.33	0.00	0.50	0.25	0.50
	3	0.32	0.00	0.48	0.24	0.50	0.40	0.00	0.67	0.33	0.50	0.33	0.00	0.50	0.25	0.50
	5	0.34	0.00	0.51	0.25	0.50	0.40	0.00	0.67	0.33	0.50	0.33	0.00	0.50	0.25	0.50
GPT-2	0	0.25	-0.43	0.33	0.22	0.32	0.18	-0.48	0.22	0.13	0.33	0.45	-0.06	0.47	0.46	0.47
	1	0.28	-0.40	0.32	0.27	0.31	0.09	-0.77	0.10	0.08	0.14	0.33	0.00	0.50	0.25	0.50
	3	0.34	0.00	0.51	0.25	0.50	0.24	0.00	0.33	0.16	0.50	0.33	0.00	0.50	0.25	0.50
	5	0.32	0.00	0.48	0.24	0.50	0.40	0.00	0.67	0.33	0.50	0.33	0.00	0.50	0.25	0.50

Table 15 ICL results for CoRT – God Class (GC), MLCQ, and GPT Dataset

LLM	Shots	CoRT – GC					MLCQ[a]					GPT Dataset[b]				
		F1	MCC	ACC	Pres	Reca	F1	MCC	ACC	Pres	Reca	F1	MCC	ACC	Pres	Reca
GPT4o-mini	0	0.59	0.250	0.61	0.64	0.61	0.27	0.110	0.35	0.28	0.32	0.60	0.611	0.66	0.60	0.66
	1	0.65	0.300	0.65	0.65	0.65	0.31	0.127	0.38	0.37	0.33	0.62	0.629	0.67	0.81	0.67
	3	0.75	0.490	0.75	0.75	0.75	0.32	0.104	0.36	0.34	0.32	0.69	0.688	0.72	0.83	0.72
	5	0.73	0.490	0.73	0.74	0.73	0.30	0.122	0.38	0.29	0.33	0.71	0.692	0.73	0.81	0.73
Gemini 1.5-flash	0	0.70	0.390	0.70	0.70	0.70	0.27	0.100	0.35	0.28	0.31	0.70	0.740	0.76	0.68	0.76
	1	0.64	0.310	0.65	0.67	0.65	0.31	0.120	0.39	0.38	0.32	0.67	0.700	0.73	0.67	0.73
	3	0.74	0.490	0.74	0.75	0.74	0.27	0.080	0.35	0.34	0.29	0.70	0.730	0.75	0.70	0.75
	5	0.76	0.520	0.76	0.76	0.76	0.30	0.100	0.37	0.33	0.31	0.74	0.780	0.80	0.70	0.80
CodeLlama-70B	0	0.42	-0.066	0.48	0.45	0.48	0.17	0.006	0.27	0.20	0.20	0.20	0.080	0.23	0.25	0.23
	1	0.44	-0.060	0.48	0.47	0.48	0.15	-0.031	0.23	0.18	0.16	0.11	-0.039	0.14	0.11	0.14
	3	0.47	0.070	0.53	0.55	0.53	0.18	0.037	0.27	0.19	0.23	0.23	0.119	0.26	0.26	0.26
	5	0.47	0.070	0.53	0.56	0.52	0.20	0.045	0.29	0.26	0.23	0.26	0.146	0.28	0.34	0.28
CodeGPT	0	0.47	0.120	0.54	0.59	0.54	0.21	-0.002	0.25	0.21	0.24	0.11	-0.030	0.14	0.18	0.14
	1	0.33	0.000	0.50	0.25	0.50	0.21	-0.050	0.24	0.21	0.22	0.04	0.000	0.16	0.02	0.16
	3	0.33	0.000	0.50	0.25	0.50	0.09	0.000	0.22	0.05	0.25	0.04	0.000	0.16	0.02	0.16
	5	0.33	0.000	0.50	0.25	0.50	0.09	0.000	0.22	0.05	0.25	0.04	0.000	0.16	0.02	0.16
GPT-2	0	0.23	-0.520	0.25	0.22	0.25	0.17	-0.080	0.22	0.20	0.20	0.11	0.050	0.20	0.08	0.20
	1	0.33	0.000	0.50	0.25	0.50	0.21	0.010	0.33	0.26	0.25	0.04	0.000	0.16	0.02	0.16
	3	0.33	0.000	0.50	0.25	0.50	0.13	0.000	0.37	0.09	0.25	0.04	0.000	0.16	0.02	0.16
	5	0.33	0.000	0.50	0.25	0.50	0.13	0.000	0.37	0.09	0.25	0.04	0.000	0.16	0.02	0.16

increase in the number of shots, as the model benefits from more examples. However, this trend is not consistent in all cases, as sometimes more shots lead to decreased performance. This phenomenon is well documented in the literature. Zhang et al. (2025) demonstrated that increasing the number of shots does not necessarily lead to improved performance. Similarly, Min et al. (2022) found that the primary role of demonstrations in ICL is to specify the task format and label space rather than to provide learning signal from individual examples, which explains why adding more examples does not always help. Furthermore, Agarwal et al. (2024) observed that even when scaling to many-shot settings, performance can plateau or degrade on certain tasks, confirming that more demonstrations do not universally translate to better results.

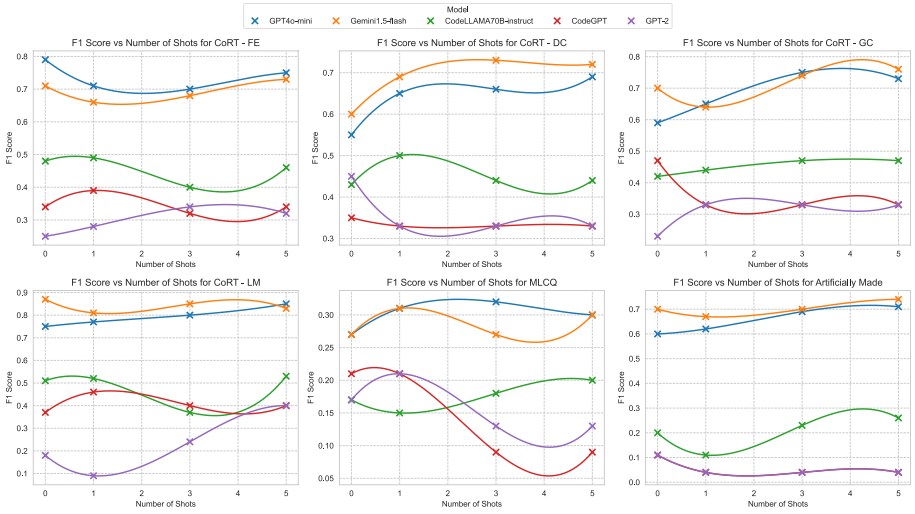

Fig. 5 ICL F1 Score Vs Shots

We identify three factors that can explain the cases where zero-shot or fewer shots outperform more shots in our experiments. First, context length dilution: as more shot examples are added, the input context grows substantially, particularly for datasets with long code samples such as God Class. With a longer context, the model may allocate less attention to the actual test sample, leading to misclassifications. This effect is more pronounced for models with limited context windows. Second, shot quality sensitivity: since shots are randomly sampled from the training set, some selected examples may be ambiguous, borderline, or even mislabeled, which can mislead the model rather than guide it. In zero-shot mode, the model relies solely on its pre-trained understanding of the task, avoiding such interference. Third, task familiarity: for well-known concepts like code smells, large LLMs such as GPT-4o may already possess sufficient knowledge from pre-training to perform the task without demonstrations. In such cases, providing examples may introduce conflicting signals that override the model's pre-existing understanding. Therefore, it is common practice to experiment with different shot configurations to determine the optimal setting for a given task.

An interesting observation from Fig. 5 is that, for most datasets, the overall performance of Gemini outperforms the other models and CodeLlama exhibits the lowest performance. This is reasonable when considering the relative sizes of the models. To compare which shot configuration yields the best performance, Fig. 6 reveals that GPT-4o-mini and CodeLlama achieve optimal results with 5 shots, whereas Gemini performs best with 3 shots. This difference could be attributed to Gemini's potential limitations in handling longer contexts. Interestingly, compact models such as GPT-2 and CodeGPT tend to prefer fewer examples, often achieving their best performance with zero-shot or one-shot prompting. This behavior may be attributed to their limited capacity to process and leverage large contextual information, making them less effective at handling longer prompts with multiple in-context examples. Moreover, in the case of GPT-4o-mini, the zero-shot setting outperforms the 1-shot setting, indicating that significant performance can be achieved with GPT-4o-mini without providing any training examples, solely by carefully formulating the task in the prompt.

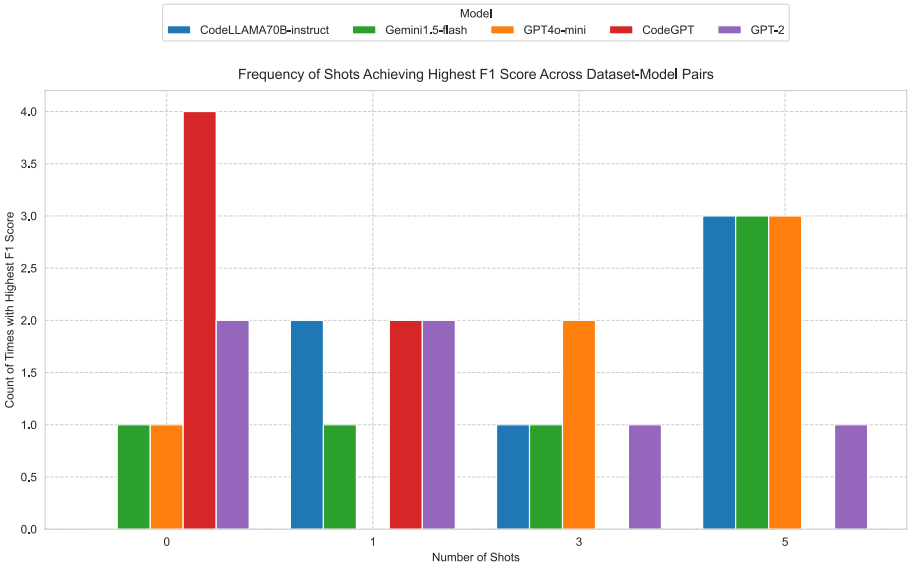

Fig. 6 The highest score with the number of shots

6.1.2 Fine-tuning

For fine-tuning, a total of 30 experimental variations were conducted. Additionally, the performance of the models prior to fine-tuning was recorded for comparison with the zero-shot setting in ICL. Training times were also collected. The results are presented in Table 16, which includes performance metrics both before and after fine-tuning.

As illustrated in Fig. 7, fine-tuning led to noticeable performance improvements. The lower the performance before training, the greater the observed improvement. Another notable observation from the line shading is the reduction in performance spread after training. Before fine-tuning, the spread was wider, likely due to the randomness inherent in the initial weights. After training, the results became more consistent across different models for the same dataset.

As expected, the performance before fine-tuning tended to assign equal probabilities to all classes. For instance, in the MLCQ dataset, which has four classes, the accuracy before training was approximately $1/4 = 0.25$, with minor deviations caused by randomness. Consequently, datasets with more classes demonstrated worse initial performance before fine-tuning.

Furthermore, Fig. 8 visualizes the F1 scores before and after fine-tuning. This figure confirms that the results before training exhibit greater variability compared to those after training. Figure 9 provides a dataset-wise comparison of performance for each model, both before and after training. UnixCoder achieved the highest or near-highest F1 scores across most datasets, except for the Artificially Constructed Dataset. Interestingly, all models achieved comparable performance after fine-tuning, highlighting the effectiveness of the process.

Table 16 Fine-tuning Results

Dataset	Model	After Fine-tuning F1	MCC	ACC	Pres	Reca	Time (s)	Before Fine-tuning F1	MCC	ACC	Pres	Reca
CoRT – Feature Envy	CodeBERT	0.89	0.78	0.90	0.91	0.87	462	0.33	0	0.50	0.25	0.50
	GraphCodeBERT	0.83	0.68	0.84	0.84	0.83	456	0.33	-0.04	0.49	0.42	0.42
	Unixcoder	0.83	0.67	0.83	0.83	0.83	492	0.40	-0.18	0.40	0.40	0.40
	CodeGPT	0.82	0.65	0.83	0.83	0.82	444	0.49	-0.01	0.49	0.49	0.49
	GPT-2	0.78	0.59	0.79	0.79	0.79	434	0.37	0.10	0.50	0.64	0.51
CoRT – Data Class	CodeBERT	0.91	0.83	0.91	0.91	0.91	470	0.33	0	0.50	0.50	0.50
	GraphCodeBERT	0.93	0.86	0.93	0.93	0.93	488	0.48	0.02	0.51	0.51	0.51
	Unixcoder	0.93	0.86	0.93	0.93	0.93	471	0.53	0.10	0.55	0.55	0.55
	CodeGPT	0.87	0.75	0.87	0.88	0.87	467	0.41	-0.11	0.45	0.43	0.45
	GPT-2	0.82	0.65	0.82	0.82	0.82	468	0.43	0.12	0.53	0.61	0.53
CoRT – God Class	CodeBERT	0.79	0.59	0.79	0.79	0.79	471	0.33	0	0.50	0.25	0.50
	GraphCodeBERT	0.81	0.63	0.81	0.81	0.81	488	0.47	0	0.50	0.50	0.50
	Unixcoder	0.82	0.66	0.82	0.83	0.82	499	0.50	0.04	0.52	0.52	0.52
	CodeGPT	0.78	0.58	0.79	0.79	0.79	465	0.38	-0.16	0.43	0.40	0.43
	GPT-2	0.71	0.42	0.71	0.71	0.71	442	0.46	0.10	0.53	0.57	0.53
CoRT – Long Method	CodeBERT	0.89	0.78	0.90	0.91	0.87	589	0.33	0	0.50	0.25	0.50
	GraphCodeBERT	0.88	0.77	0.90	0.91	0.86	584	0.45	-0.03	0.46	0.46	0.47
	Unixcoder	0.89	0.81	0.91	0.93	0.87	602	0.36	-0.24	0.38	0.36	0.37
	CodeGPT	0.88	0.77	0.90	0.90	0.87	542	0.38	-0.20	0.41	0.40	0.39
	GPT-2	0.86	0.74	0.89	0.90	0.84	523	0.53	0.31	0.71	0.85	0.57
MLCQ	CodeBERT	0.79	0.72	0.80	0.80	0.78	512	0.07	0	0.28	0.07	0.25
	GraphCodeBERT	0.78	0.73	0.81	0.81	0.77	531	0.17	-0.07	0.21	0.17	0.19
	Unixcoder	0.81	0.72	0.80	0.81	0.76	533	0.24	0.05	0.25	0.30	0.31
	CodeGPT	0.71	0.63	0.74	0.75	0.70	509	0.18	-0.05	0.20	0.23	0.21
	GPT-2	0.69	0.59	0.71	0.75	0.66	511	0.17	0.01	0.31	0.21	0.23
GPT Dataset	CodeBERT	0.926	0.914	0.927	0.937	0.927	191	0.04	0	0.16	0.02	0.16
	GraphCodeBERT	0.970	0.970	0.970	0.980	0.970	195	0.12	-0.01	0.15	0.19	0.15
	Unixcoder	0.940	0.930	0.940	0.940	0.940	202	0.12	0	0.16	0.15	0.16
	CodeGPT	0.680	0.620	0.690	0.670	0.690	123	0.09	-0.09	0.09	0.12	0.09
	GPT-2	0.660	0.600	0.670	0.680	0.670	113	0.09	-0.02	0.15	0.15	0.15

6.2 Statistical analysis

In the results section, we briefly discussed the outcomes of the two treatments independently. In this section, we perform a comparative analysis to evaluate both treatments and address the stated null hypothesis. The results of the statistical tests conducted are summarized in Table 17 where W_+ is the number of ranked positive, W_- is the number of ranked negative, where the difference was calculated as $finetuning - ICL$ so a positive rank means that fine-tuning was higher and a negative one indicates that ICL was higher, and W_c is the critical value from the table based on the sample size and confidence interval. A detailed discussion is provided in the subsequent sections. Our sample size is 6 in all cases, and the confidence interval is set to 0.05. We used the Wilcoxon Signed-Rank Test since our data is not normally distributed, our experiment is with one factor and two treatments, and our values are paired by the dataset, since each technique was applied to the same datasets.

6.2.1 H_0^1 H0^1 : performance

To evaluate performance, the average F1 score for each dataset was calculated for both treatments: ICL and fine-tuning. The results are visualized in Fig. 10, where it is clear that fine-tuning consistently achieved better results.

To test the null hypothesis H_0^1, we used the Wilcoxon Signed-Rank Test because the data points were not normally distributed. The difference was taken as (fine-tuning - in-context). The test yielded a p-value of 0.0156, which is less than the significance threshold of 0.05. **Therefore, with a 95% confidence level, we reject** H_0^1. Also, 21 ranks were positive, with

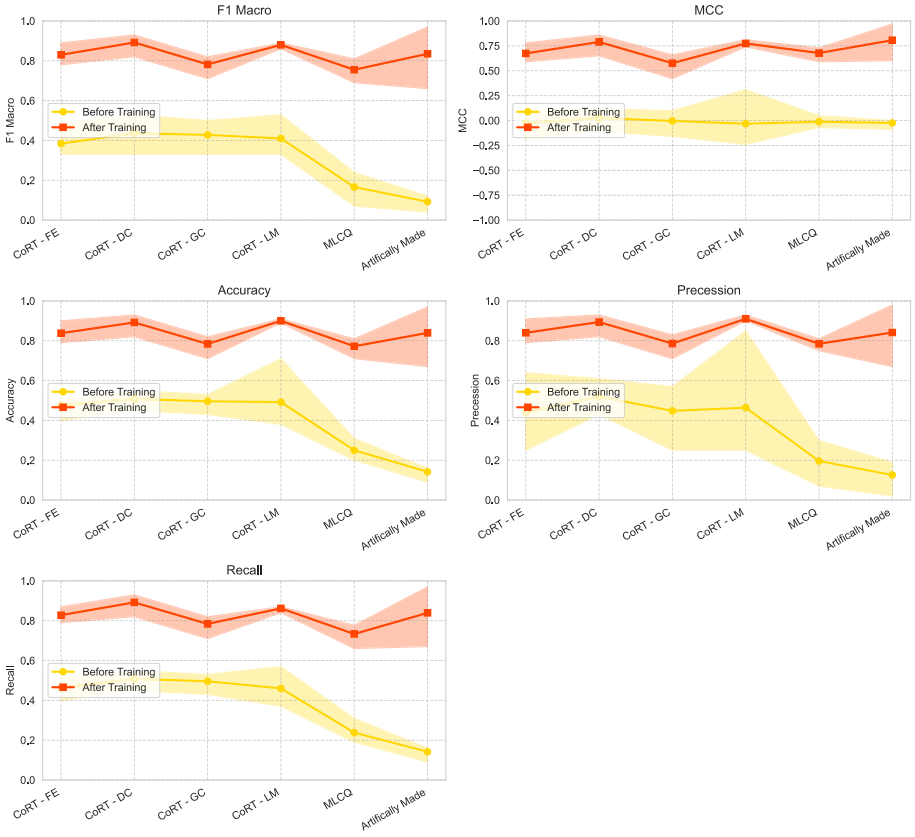

Fig. 7 Fine-tuning results before and after training

a sample size of 6 and a confidence level of 0.05. The critical value is 2, so we reject the null hypothesis.

Since the null hypothesis is rejected, we accept the alternative hypothesis H_a^1 sine W_+ is 21 and W_- is zero, and the mean difference is 0.395, which is positive, which means that the performance, as measured by the F1 score for code smell detection, is higher with fine-tuning than with In-Context Learning. Moreover, from Fig. 10, it is clear that fine-tuning outperforms ICL in terms of F1 score.

For a broader perspective, comparing the fine-tuning performance line to the upper bound of the shaded area in the ICL results (representing the highest values obtained using ICL) reveals that fine-tuning still performs better. However, the performance gap between the two methods is narrower for some datasets (e.g., CoRT - God Class), where ICL achieved similar or occasionally better results. This indicates that, with appropriate optimization and model selection, ICL can approach the performance of fine-tuning but is unlikely to surpass it.

Furthermore, the wider shadow in the ICL results indicates a higher variance, suggesting that performance is more sensitive to factors such as the choice of model and configuration. This observation is further supported by Fig. 11, which shows significant variability in ICL results. These findings suggest potential opportunities for improvement in future studies, such as exploring different optimization techniques or model configurations.

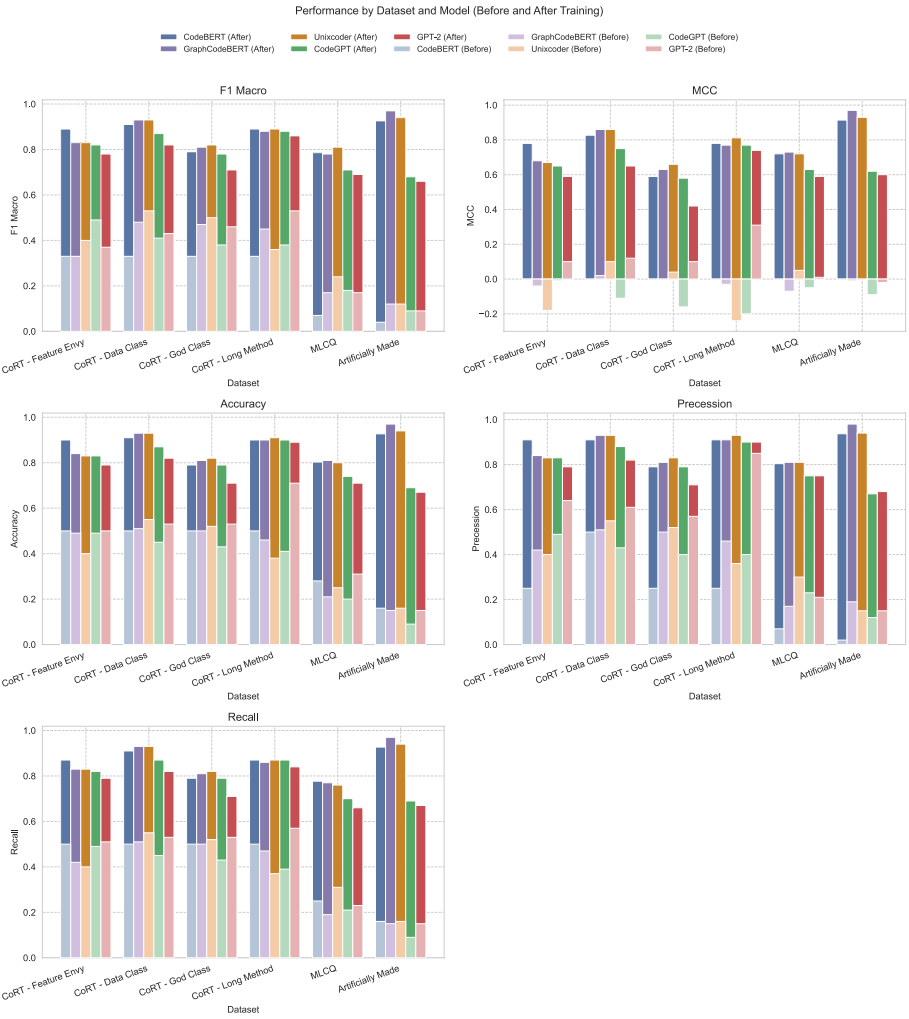

Fig. 8 Box blot for the fine-tuning F1 score

6.2.2 $H_0^2 H0^2$: training time

For training time, the results for fine-tuning are reported in Table 16, where the average time taken per dataset is calculated. In contrast, ICL is considered a lazy training approach since it does not involve updating the model's weights. In other words, the LLM is pre-trained and ready for inference, and the test set is directly evaluated.

While some pre-testing techniques, such as generating embeddings to retrieve the most similar samples during testing, might be considered part of the training time in certain scenarios, no such techniques were employed in this study. Therefore, the training time for all ICL experiments is considered zero.

To test the null hypothesis H_0^2, we used the Wilcoxon Signed-Rank Test, which resulted in a p-value of 0.0078. Since this is less than the significance threshold of 0.05, **we reject**

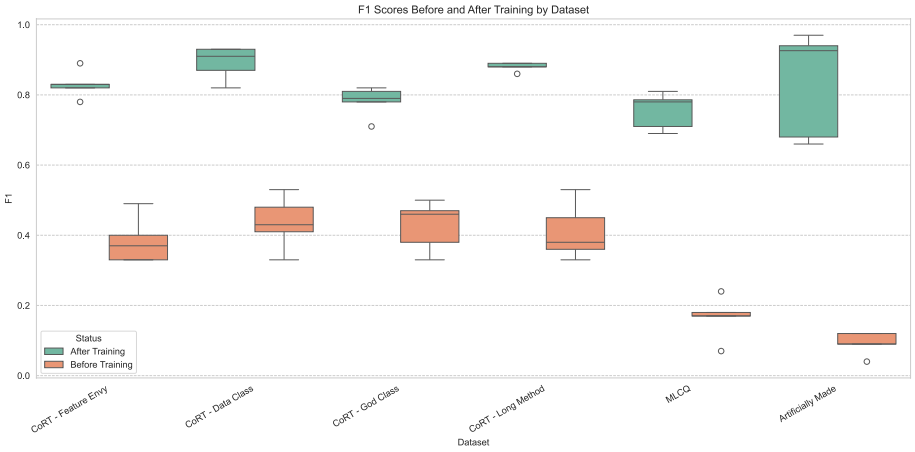

Fig. 9 Performance by dataset and model metrics

Table 17 Statistical test results for different factors

Hypothesis	Factor	Mean Diff.	W_+	W_-	W_c	n	p-Value
H_0^1	F1 Score	0.395	21	0	2	6	0.0156
H_0^2	Time	442.57	21	0	2	6	0.0078
H_0^3	Training Samples	1041.25	21	0	2	6	0.0078

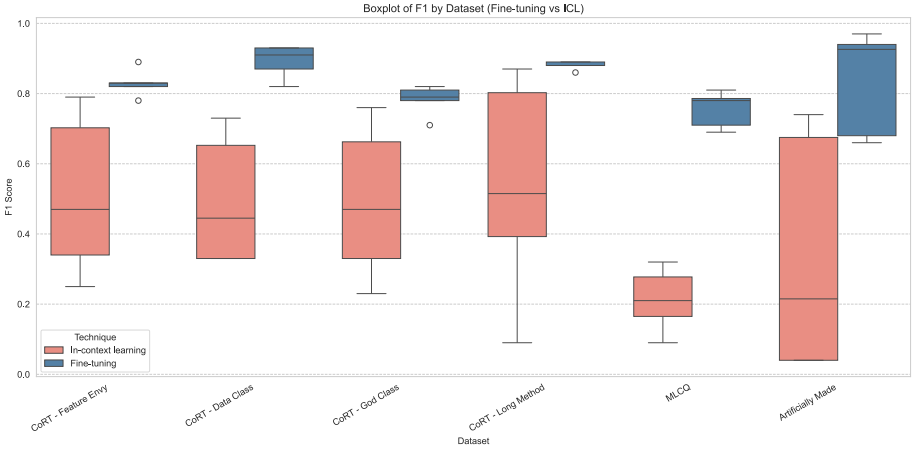

Fig. 10 Fine-tuning Results Vs ICL Results

the null hypothesis H_0^2 with a 95% confidence level. Moreover, as can be seen in Table 17 21 ranks were positive, with a sample size of 6 and confidence level of 0.05 the critical value is 2, so we reject the null hypothesis. Regarding the alternative hypothesis H_a^2, it is evident that fine-tuning requires more time as ICL involves no training and W_+ is greater than W_- ; also, the mean difference is positive. Thus, we accept H_a^2that the time taken for training,

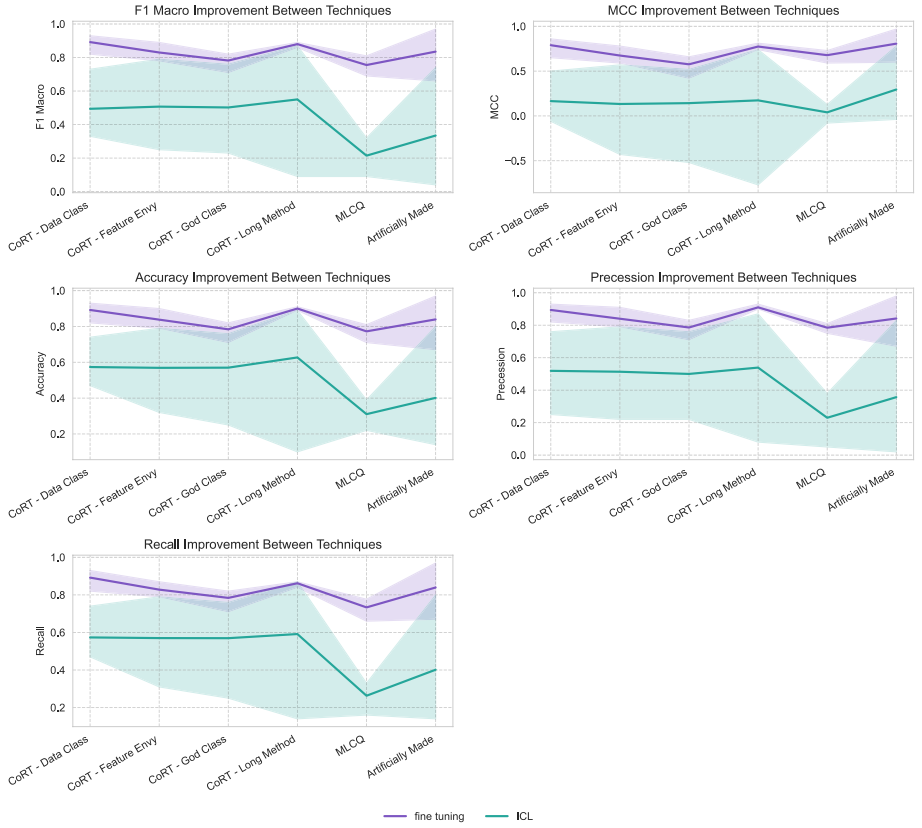

Fig. 11 F1 Score Box Plot for Fine-tuning Results Vs ICL Results

measured in seconds, for code smell detection using fine-tuning is significantly higher than using In-Context Learning.

6.2.3 $H_0^3H0^3$: training samples

Regarding training samples, fine-tuning requires the entire training set. In contrast, for ICL, the number of training samples needed is less straightforward to quantify. ICL can operate in a zero-shot setting, where no training samples are required. For experiments with few-shot settings, we used a maximum of 5 shots, which were randomly drawn from the training set. Notably, these shots could also be fixed or drawn from a much smaller subset of the training set.

To estimate the average number of training samples used in ICL, we considered the arithmetic mean of the number of shots across all settings:

$$\text{Average shots} = \frac{0+1+3+5}{4} = 2.5$$

We performed the Wilcoxon Signed-Rank Test to evaluate the hypothesis and obtained a p-value of 0.0078, which W_+ is equal to 21 with W_c from the table equals to 2, so with a p-value below the significance threshold of 0.05, **we reject the null hypothesis** H_0^3 with a 95% confidence level.

It is evident that fine-tuning typically requires a larger number of training samples compared to ICL. Also, $W_+ > W_-$ we accept the alternative hypothesis H_a^3, which asserts that the number of training samples required for fine-tuning is significantly higher than for In-Context Learning. Moreover, another evidence to accept H_a^3 is that the mean difference is 1041.25, which is positive and indicates that fine-tuning needs more training samples to train.

7 Discussion and Implications

This section addresses the stated research questions (RQs) in Table 1 to determine which technique is more suitable for the task. The answers are supported by the statistical analyses conducted in the previous section.

7.1 RQ1: Which technique demonstrates better performance in the task?

To answer this question, we refer to the results of H_0^1. Our findings consistently show that fine-tuning achieves better performance. While our statistical tests specifically focused on the F1 score, all other reported metrics also indicate that fine-tuning outperforms ICL, as demonstrated in Table 11.

When comparing the maximum results obtained for each dataset using ICL, the performance gap with fine-tuning is not substantial. However, it is evident that LLMs struggle the most with the MLCQ dataset, which involves a multi-class classification task. One possible reason for this is the varying definitions of each code smell. In the case of ICL, the model relies solely on the encapsulated knowledge stored in its weights, whereas in fine-tuning, the model explicitly learns these definitions during training. This learning process helps mitigate confusion when dealing with multiple options in multi-class problems.

> **Summary:** If achieving better performance is your primary objective, fine-tuning should be favored over In-Context Learning.

7.2 RQ2: Which technique can generalize better?

To address this research question, we analyzed generalizability from three perspectives: 1) Noise Sensitivity, 2) Flexibility to Change, and 3) Cross-Dataset Validation.

7.2.1 Noise Sensitivity

We examined the predictions made by both techniques. Consider the following test sample from the CoRT - Feature Envy dataset:

```
public void eval ( ) { out . value = org . apache . drill . exec . expr
. fn . impl . HashHelper . hash64 ( in . value , 0 ) ; }
```

This method primarily accesses the `out.value` attribute rather than its own class attributes, suggesting it contains a Feature Envy smell. However, the true label for this sample, as annotated in the original dataset, is 0, indicating the absence of a Feature Envy smell. We believe this sample represents noise in the dataset.

Interestingly, the fine-tuned model, having learned from similar patterns in the same dataset, predicted the label as 0 (non-smelly). Conversely, the ICL model identified the sample as containing a Feature Envy smell, irrespective of the noise present in the training set.

This difference can be attributed to the distinct nature of the two approaches. Fine-tuning is predominantly learned from the training dataset, making it sensitive to annotations and potential noise. In contrast, ICL leverages the pre-trained knowledge embedded in the model's weights, derived from extensive and diverse data. As a result, ICL often generalizes better by relying on the semantic understanding of terms like *Feature Envy*. In contrast, fine-tuning tends to treat labels as numerical categories, potentially diminishing the relevance of their semantic meaning.

7.2.2 Cross-dataset validation

We conducted an additional experiment to evaluate the generalization capability of both techniques. This experiment was restricted to specific datasets due to the inherent differences in class structures. We selected a subset of samples from the MLCQ dataset (originally not binary) and reformulated it as a binary classification problem for the 'Data Class'.

First, we tested a fine-tuned model trained on the CoRT 'Data Class', achieving an initial test score of 93%. When the same model was evaluated on the binary MLCQ dataset, its performance dropped significantly to 82%. This result underscores a limitation of fine-tuning: its inability to generalize effectively across datasets, as it predominantly learns patterns specific to the training set.

Next, we applied the ICL technique on the same binary MLCQ dataset, using shots sampled from the CoRT 'Data Class' training set. Interestingly, the model's performance improved even when exposed to shots from a different dataset. This behavior indicates that ICL enables the model to learn the task itself rather than memorize patterns or specific samples from the training data. Although we did not apply this procedure to all datasets due to compatibility issues, this finding supports the assertion that ICL can generalize better than fine-tuning. Furthermore, as Mueller et al. (2023) demonstrated, ICL tends to generalize well for certain tasks. To elaborate, ICL involves providing examples to LLMs so that they can recognize the underlying task and apply the learned pattern to a new test sample. Since LLMs already encapsulate extensive knowledge acquired through pre-training on massive corpora, the main challenge lies in adapting this knowledge to follow a specific task (in our case, code smell detection), which originally motivated the development of ICL (Radford et al. 2019).

For instance, if the model is given an example of a code snippet containing a *God Class* and labeled accordingly, it can infer that the task involves identifying code smells. When subsequently presented with a new test sample exhibiting a different smell, such as a *Data Class*, the model can generalize from the prior examples and correctly classify it. The funda-

mental idea behind providing such examples, or "shots," is to establish recognizable input–output pairs that enable the model to discern and replicate the pattern in new contexts.

7.2.3 Flexibility

Fine-tuned models, tailored to specific datasets, have fixed architectures with output layers designed for predefined classes, making them inflexible to accommodate new smells. In contrast, ICL offers flexibility; adding a new smell requires only rewriting the prompt, potentially with a few examples or even none in a zero-shot setting. This makes ICL advantageous for tasks requiring adaptability to new categories.

> **Summary:** Fine-tuning is advantageous when working with domain-specific datasets, such as those tailored to a particular organization or application. It enables the model to capture specific patterns, coding styles, and the annotators' interpretations of smells. However, fine-tuned models are constrained by their sensitivity to noise and limited ability to generalize across datasets. In contrast, ICL demonstrates superior generalization capabilities by leveraging pre-trained knowledge to adapt effectively to new and unseen data. ICL excels in tasks requiring flexibility, as it allows easy incorporation of new smells through prompt engineering without the need for retraining. Therefore, while fine-tuning is well-suited for specialized use cases, ICL is more appropriate for building robust, adaptable classifiers for general-purpose scenarios and evolving requirements.

7.3 RQ3: Which of these two techniques is preferred in terms of the available data?

This question is critical as it helps determine the preferred technique based on the availability of data. As demonstrated in the statistical test of H_a^3, fine-tuning consistently requires more data to achieve optimal performance. In fine-tuning, the pre-trained model iterates over the entire training set for multiple epochs until convergence to a stable minimum of the loss function. Conversely, ICL is highly efficient in terms of training data, as it can be applied without any labeled training samples.

To ensure a fair comparison in scenarios where labeled data is unavailable, we compared the results of zero-shot learning in ICL with the predictions of pre-trained models before fine-tuning. The comparison, illustrated in Fig. 12, reveals that zero-shot learning consistently outperforms pre-trained models without fine-tuning. The performance of pre-trained models is nearly random, with worse results as the number of classes increases. In contrast, ICL, even without any training examples, is capable of understanding the task and the meaning of each code smell, yielding reasonable predictions.

Moreover, even when labeled data is not much available, fine-tuning can be improved with techniques like Active Learning (Cohn et al. 1994), which optimizes supervised learning by selectively labeling the most informative samples, thus enhancing performance with fewer training examples.

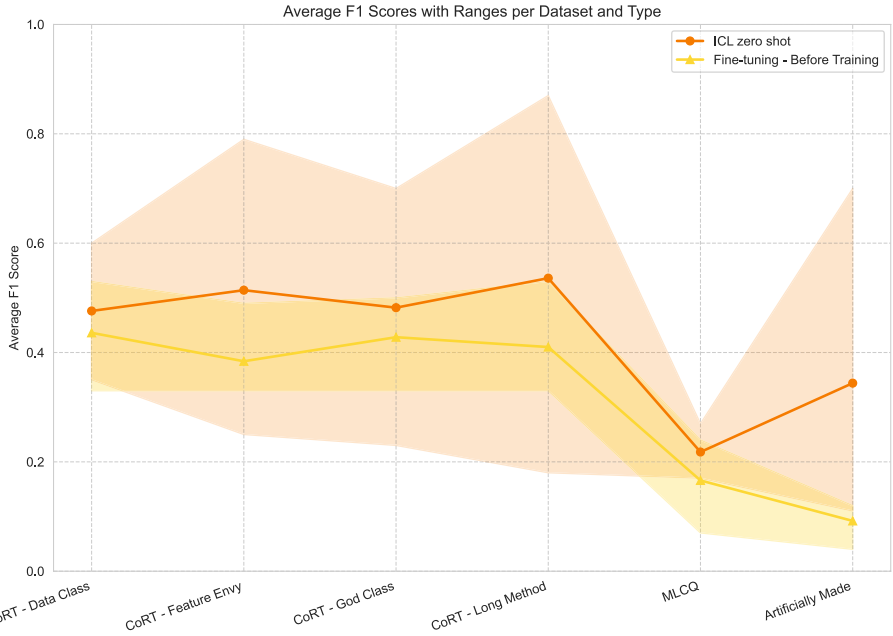

Fig. 12 Zero-shot vs before training

Summary: As a general rule, if you have sufficient labeled training data and achieving high performance is critical, fine-tuning should be preferred over ICL. However, if labeled data is unavailable—such as for programming languages with limited resources—you can still achieve good performance with zero-shot learning. Additionally, with a small dataset, a few-shot approach can be used to further enhance ICL performance.

7.4 RQ4: Which of these two techniques is preferred in terms of available hardware resources and time?

To address this question, we refer to the findings for H_0^2, where it was established that fine-tuning consistently requires more time for training. Additionally, loading an LLM for inference only, as in the case of ICL, demands fewer hardware resources compared to fine-tuning, which involves updating model weights. Fine-tuning requires additional memory for tasks such as storing gradients and performing optimizations, significantly increasing resource consumption.

For instance, in our experiment, we were unable to fine-tune CodeLlama-70B due to hardware limitations, but we were able to use it for inference in an ICL setting. Conversely, ICL can be performed using publicly available APIs, such as OpenAI's GPT API and Google Gemini, which we utilized in this study. These APIs eliminate the need for local computational resources and can be easily integrated into workflows.

Moreover, fine-tuning often requires technical expertise in AI and model optimization, making it less accessible to non-experts. On the other hand, using APIs for ICL is relatively straightforward and more user-friendly for the general public. Despite these advantages, it is important to note that while fine-tuning requires more resources and expertise, it can deliver superior performance, as demonstrated in this study.

Another important aspect to consider when dealing with resources is the *cost*. Evaluating the cost requires examining the issue from two different perspectives. Generally, fine-tuning LLMs demands significantly higher computational resources (Dettmers et al. 2023) compared to performing inference, as is the case in in-context learning. During fine-tuning, the training process involves storing various computational and gradient-related components necessary for backpropagation and updating model weights. In contrast, ICL involves only inference, often referred to as a forward pass, which does not require weight updates. From this standpoint, fine-tuning is more computationally expensive.

However, on the other hand, the models used for fine-tuning, particularly for specific tasks such as code smell detection as in our case, are typically much smaller. With sufficient high-quality data, these smaller models can effectively learn the target task through training. Conversely, in ICL, the model size and context length play crucial roles. For instance, in our experiments, fine-tuning was performed using BERT-based models containing fewer than 0.5B parameters, while ICL relied on large-scale models such as GPT-4o-mini, Gemini, and LLaMA, with the smallest among them containing around 70B parameters (Ai 2024).

To provide a quantitative perspective, we estimated the monetary cost of running our ICL experiments based on the token consumption and the published API pricing at the time of the experiments. Token counts were computed using the `tiktoken`[10] library applied to the actual prompts and code samples used in each experiment. The per-token pricing used was \$0.15/1M input and \$0.60/1M output tokens for GPT-4o-mini[11], and \$0.075/1M input and \$0.30/1M output tokens for Gemini-1.5-Flash[12], based on the rates at the time of experimentation. For CodeLlama-70B, which was deployed locally via Ollama in our experiment, we provide a hypothetical cost estimate based on the median pricing for comparable 70B-parameter models offered through API providers such as Together.ai[13] (\$0.58/1M input and \$0.78/1M output tokens). Table 18 summarizes the estimated costs.

The total estimated cost for running all 24 ICL experiment variations per model (6 datasets $\times$ 4 shot settings) was approximately \$4.01 for GPT-4o-mini, \$2.00 for Gemini-1.5-Flash, and an estimated \$15.49 for CodeLlama-70B if it were accessed through an API provider. It is worth noting that CodeLlama-70B is an open-source model and can be deployed locally using tools such as Ollama, as was done in our experiment, in which case no API cost is incurred. Likewise, the two compact ICL models (CodeGPT and GPT-2) were run locally and therefore incurred no API cost. However, local deployment requires sufficient hardware resources (e.g., high-VRAM GPUs), and the pricing for such models through API providers varies over time as newer and more efficient models are released. For fine-tuning, the 30 experiment variations (6 datasets $\times$ 5 models) required a total of approximately 4 hours of GPU time on NVIDIA T4 hardware, which was provided free of charge through Kaggle. At

[10] https://pypi.org/project/tiktoken/

[11] https://openai.com/api/pricing/

[12] https://ai.google.dev/gemini-api/docs/pricing

[13] https://www.together.ai/pricing

Table 18 Estimated monetary cost (USD) of ICL experiments by dataset, model, and number of shots

	GPT-4o-mini				Gemini-1.5-Flash				CodeLlama-70B*			
Dataset	0	1	3	5	0	1	3	5	0	1	3	5
CoRT – God Class	0.159	0.343	0.711	1.080	0.079	0.171	0.356	0.540	0.612	1.325	2.749	4.174
CoRT – Data Class	0.045	0.075	0.134	0.193	0.023	0.037	0.067	0.096	0.174	0.288	0.516	0.744
CoRT – Feature Envy	0.015	0.024	0.040	0.057	0.008	0.012	0.020	0.028	0.059	0.090	0.154	0.218
CoRT – Long Method	0.018	0.032	0.060	0.088	0.009	0.016	0.030	0.044	0.069	0.123	0.230	0.337
MLCQ	0.068	0.137	0.276	0.415	0.034	0.069	0.138	0.207	0.261	0.530	1.066	1.603
GPT Dataset	0.004	0.007	0.012	0.018	0.002	0.003	0.006	0.009	0.014	0.025	0.047	0.068
Total	$4.01				$2.00				$15.48*			

*CodeLlama-70B was run locally; costs are hypothetical estimates based on comparable 70B model API pricing

typical cloud on-demand pricing of approximately $0.35/hour per GPU on Google Cloud[14], the estimated fine-tuning cost would be approximately $1.72 (2 GPUs × 2.5 hours × $0.35/hour).

Therefore, calculating the overall cost is not straightforward and depends on how the techniques are applied in practice. If we fix the model and compare the two techniques directly, fine-tuning will generally be more computationally expensive, as it requires storing gradients, optimizer states, and performing backpropagation, all of which demand more GPU memory and longer processing time compared to the forward pass used in ICL inference. However, if we compare the techniques as typically applied in practice, fine-tuning may in fact be less costly in monetary terms. This is because fine-tuning is commonly performed on smaller, task-specific models. In our experiments, the fine-tuning models (CodeBERT, GraphCodeBERT, UniXcoder) contain fewer than 0.5B parameters, whereas the ICL models range from 70B to over 100B parameters. Note that in this paper we used two compact models (GPT-2 and CodeGPT) for controlling the experiment, however they are not commonly used these days for ICL. As a result, the total ICL cost exceeded the estimated fine-tuning cost ($1.72) for all three ICL models, despite ICL requiring no training phase. The cost of ICL scales with the number of input tokens, which grows substantially with the number of shots and with longer code samples such as those in the God Class dataset. Moreover, it is worth noting that many LLM providers offer API-based access to their models at per-token pricing, which may be more cost-effective than hosting large models locally.

> **Summary:** ICL is generally easier to perform, as it requires no training time and fewer computational resources. It can even be conducted using readily available APIs. However, the monetary cost of ICL can exceed that of fine-tuning when applied to datasets with long code samples, as API costs scale with token consumption. Fine-tuning might be the better choice if the highest possible performance is critical for the task and sufficient resources and time are available.

7.5 What technique to use?

Based on the previous discussion, we summarize the considerations for choosing between the two techniques. As is often the case in software engineering, the answer to this question is: "It depends." Selecting the appropriate technique depends on various factors outlined below to aid in decision-making. Figure 13 illustrates a decision tree that can be followed to identify the best technique based on these factors.

Our findings indicate that fine-tuning typically yields better results but requires a substantial amount of labeled data. Therefore, the availability of labeled data becomes a crucial factor.

If labeled data is highly available, such as for widely-used programming languages like Java, fine-tuning is generally the better choice, provided sufficient time and computational resources are also available. On the other hand, if labeled data is unavailable, ICL might be the only possible option unless a new labeled dataset can be constructed.

In cases where labeled data is limited, the decision depends on the task's performance requirements. If high performance is critical, fine-tuning can still be considered, using tech-

[14] https://cloud.google.com/compute/gpus-pricing

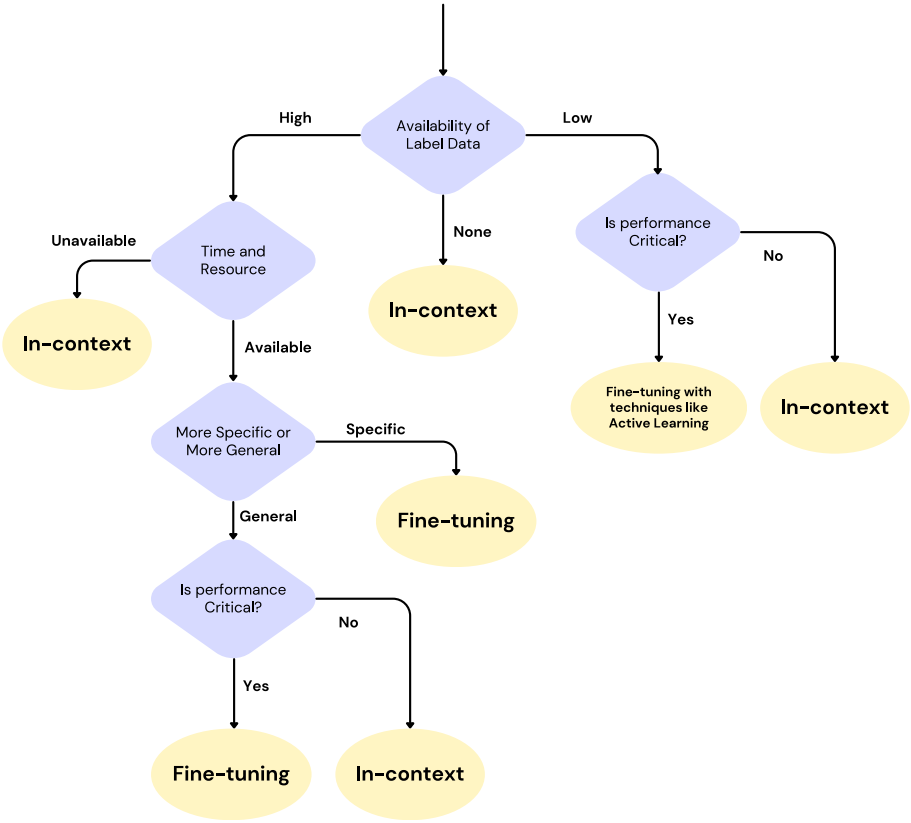

Fig. 13 Technique decision tree

niques such as Active Learning or Data Augmentation to enhance performance with fewer training samples. However, if performance requirements are less stringent, ICL may be preferable due to its lower data dependency.

The intended use case becomes the determining factor if both labeled data and resources are available. Fine-tuning is more suitable for domain-specific classifiers, such as those tailored to a specific company or dataset, as it allows the model to learn domain-specific patterns and annotations. In contrast, for general-purpose classifiers, where performance is not the highest priority, ICL may generalize better and offer more flexibility.

Table 19 summarizes the comparison between the two techniques, highlighting their trade-offs based on the discussed factors.

8 Threats to validity

This study may face various threats to its validity. This section highlights these threats and discusses the measures taken to mitigate their impact.

Table 19 Comparison of in-context learning and fine-tuning techniques

Technique	Performance	Data Consumption	Time Taken	Generalizability
In-Context Learning	Less	Low	Low	Better
Fine-tuning	Better	High	High	Less (Better in domain-specific)

8.1 External threats

External threats to validity primarily affect the generalizability of the experimental results. We made efforts to ensure our findings are as generalizable as possible, but certain challenges remain.

8.1.1 Java datasets

One potential threat is the reliance on Java datasets, which may limit the generalizability of our results to other programming languages. However, Java was chosen because it is one of the most extensively studied programming languages in the literature. To mitigate this limitation, we constructed an artificial dataset that includes additional programming languages such as C++, Python, and JavaScript, thereby broadening the scope of our analysis.

8.1.2 Choice of LLMs

The choice of LLMs used in ICL experiments may also impact the generalizability of the study. To address this, we conducted experiments with five different LLMs. Furthermore, we selected diverse models to minimize bias: GPT-4o-mini and Gemini are closed-source models, while CodeLlama, CodeGPT, and GPT-2 are. Additionally, CodeLlama and CodeGPT are primarily trained on code-specific data, whereas GPT-4o-mini, and Gemini, and GPT-2 were trained on general-purpose data. We also included models with varying parameter sizes from compact models with 124M parameters to large models with billions of parameters to ensure a broad representation of capabilities.

Moreover, some of the models selected for the ICL experiments, such as GPT-2 and CodeGPT, are relatively small and older than more recent language models so they might not fully reflect the performance of ICL. However, these models were chosen because they support text generation for ICL and can also be fine-tuned using our available hardware resources, enabling a consistent comparison between ICL and fine-tuning. To complement these compact models, three additional large-scale models were included in the ICL evaluation.

8.2 Conclusion threats

Conclusion validity refers to the threats that may affect the reliability of the statistical tests performed to evaluate our hypotheses.

8.2.1 Wilcoxon signed-rank test

We utilized the Wilcoxon Signed-Rank Test to evaluate all null hypotheses, which could potentially threaten our results' validity. The rationale for selecting this test lies in the nature of our data, which is not normally distributed. Also, our data is paired by dataset since each technique was applied to each dataset, so a paired test is suitable in our case. Alternative tests, such as the paired t-test, would not be appropriate in this context due to their reliance on the assumption of normality. Moreover, usually, the risk comes from applying parametric tests on non-normally distributed data, where in our case we used a non-parametric test, so we do not have that risk.

8.2.2 Small Sample Size (n)

One potential limitation of our analysis arises from the small sample size ($n = 6$) used to evaluate the null hypothesis. A sample size this small can influence the statistical significance of the results. However, it is important to note that these six values represent the means derived from 120 independent experiments conducted for ICL and 18 independent experiments conducted for fine-tuning. Thus, although the direct sample size is relatively small, it aggregates data from a larger experimental base. Furthermore, the observed p-values are Table 17 are significantly smaller than the conventional significance threshold of 0.05. This provides strong evidence to confidently reject the null hypothesis despite the small sample size.

8.3 Construct validity

Construct validity refers to the concern of whether we are accurately measuring what we intend to measure in the study.

8.3.1 Using F1 score for effectiveness

One potential threat is the reliance on the F1 score as the primary metric for evaluating the dependent variable, which is the model's effectiveness in this case. The F1 score was chosen because it is widely used in machine learning and deep learning applications. Additionally, the F1 score is particularly advantageous when dealing with imbalanced datasets, as it provides a balanced measure of precision and recall, unlike accuracy, which can be overly sensitive to class imbalances.

Although the F1 score was the primary metric, we also reported other commonly used metrics in AI, such as Matthews Correlation Coefficient (MCC), accuracy, recall, and precision, to provide a comprehensive evaluation. According to Alazba et al. (2023), the F1 score is one of the most frequently used evaluation metrics in code smell detection tasks, supporting its appropriateness for this study.

8.3.2 Using time and number of samples for efficiency

Another potential threat arises from our reliance on training time and the number of training samples as measures of model efficiency. Efficiency is a broad term that can encompass

various factors, such as energy consumption or computational resource usage. To mitigate this threat, we used two measures–training time and number of samples–to capture different aspects of efficiency.

Furthermore, since this study is conducted from the perspective of researchers, we believe that training time and the availability of labeled samples represent the most significant challenges in training deep learning systems. These aspects also highlight key distinctions between the two treatments applied in this study.

8.4 Internal validity

Internal threats to validity refer to factors that could affect the reliability of the conclusion that changes in the dependent variable are solely due to the applied treatments.

8.4.1 Fixing ICL parameters

One factor that could influence the results for ICL is the selection of model parameters, such as temperature, `top_p`, and `max_tokens`. Since there is no universally agreed-upon best value for these parameters, we mitigated subjectivity by using the default values provided by OpenAI for GPT-4o-mini. These parameter settings were then fixed and applied consistently across all other models, ensuring they were treated as independent variables in the experiment.

8.4.2 Prompt design

The design of the prompt used in ICL experiments could also impact the results. Numerous studies propose techniques for crafting effective prompts. To maintain the scope of our experiment, we employed a simple prompt that framed the classification task following the guidelines proposed by Radford et al. (2019).

8.4.3 Hyperparameters in fine-tuning

The choice of hyperparameters during fine-tuning could affect the outcomes. To address this, we conducted hyperparameter tuning using a validation set, and the selected hyperparameters were used consistently in the experiments. Certain parameters, such as the learning rate, were fixed, and their selection was justified in the planning phase.

8.4.4 Using different LLMs

Another potential threat is the use of partly some different LLMs for ICL and fine-tuning. For the large models, this choice was necessary because high-capacity LLMs used in ICL are often closed-source and cannot be fine-tuned due to accessibility or resource constraints. Conversely, the models used in fine-tuning are those commonly employed in fine-tuning studies.

Despite using different models, the LLMs used in ICL were larger than those used in fine-tuning, yet fine-tuning still achieved better performance. Therefore, the superior per-

formance observed in fine-tuning is unlikely to be attributed to model capacity, minimizing the risk of bias in this regard.

Moreover, most of the models used for fine-tuning are code-specific models, which might suggests a potential bias toward producing higher results. However, the choice of model is influenced by the nature of the task itself, and we followed the models that are commonly used for fine-tuning in code related tasks (Alazba et al. 2023). In addition, for in-context learning, we employed one code-oriented language model, CodeLlama, which did not outperform GPT-4o-mini or Gemini. This indicates that there is no inherent bias for code language models, and the observed differences in performance are due to the technique itself rather than the type of model. Furthermore, GPT-4o-mini and Gemini are general-purpose models trained on vast amounts of data that also include programming-related content.

To further mitigate this threat, we included two compact (CodeGPT and GPT-2), and they were used for both ICL and fine-tuning. The results of the two models confirmed the same pattern observed in the other models as well: fine-tuning consistently outperformed ICL across all datasets.It is worth noting that GPT-2 and CodeGPT are relatively compact models and may not fully capture ICL capabilities. However, they are complemented by three additional large-scale models for ICL. This provides empirical evidence that the observed performance difference is attributable to the learning technique rather than to the choice of model.

8.4.5 Artificially generated dataset

A potential threat to the validity of our study is that one of the datasets was artificially generated using ChatGPT and later evaluated using the same model. While this concern is understandable, it does not undermine the overall reliability of our results. The artificially generated dataset serves only as a complementary resource alongside two large datasets collected from real open-source projects. Its primary purpose is to mitigate external validity threats and enhance the generalizability of our findings across different programming languages and code smells. Furthermore, the LLMs exhibited similar performance patterns on this dataset as on the other two, suggesting that no model-specific bias is present. Notably, Gemini outperformed ChatGPT on this dataset, further indicating the absence of bias toward ChatGPT.

9 Conclusion and future work

This study aimed to compare the effectiveness and efficiency of using ICL and fine-tuning for the task of code smell detection. Various LLMs, ranging from closed-source to open-source models, were evaluated on diverse datasets using these two techniques. Three code smell detection datasets were considered, focusing on code smells such as Feature Envy, Long Method, Large Class, and Long Parameter List across multiple programming languages, predominantly Java, Python, C++, and JavaScript. We formulated 4 research questions about performance, generalizability, training time, and training samples. Also, for measurable aspects like performance in F1 score, training time in seconds, and training

samples in samples, we had a null hypothesis for each with two alternatives. We were able to reject all null hypotheses with a confidence level of 95% and synthesize the results to answer all stated RQs.

For ICL, different numbers of shots (0, 1, 3, and 5) were used, and each model was evaluated on the test set. In the case of fine-tuning, hyperparameter optimization was conducted prior to evaluation on the test set. Fine-tuning consistently demonstrated superior performance in terms of F1-score across all datasets, with a mean improvement of 26% compared to ICL. However, ICL exhibited better generalization and lower sensitivity to noise, indicating its potential to detect new types of code smells that were not present in the training datasets.

A notable limitation of fine-tuning was its decline in performance during cross-dataset validation, whereas ICL was capable of leveraging examples drawn from other datasets. ICL was also found to be significantly more efficient in terms of data requirements, requiring an average of 1041.25 fewer samples than fine-tuning. Thus, when labeled training data is abundant and high accuracy is paramount, fine-tuning is the more suitable approach. Conversely, in scenarios where training data is limited–a common challenge for less-studied programming languages–ICL, even with few-shot learning, offers acceptable performance. For small datasets, ICL remains the preferred choice due to its efficiency.

In addition to data efficiency, ICL also outperformed fine-tuning in terms of training time, with an average time savings of 459.39 seconds. Given its ability to function without requiring any training, ICL is particularly advantageous in contexts with limited hardware resources or strict time constraints. However, this efficiency may not always translate to acceptable performance for the specific task of code smell detection. The choice of technique–ICL or fine-tuning–ultimately depends on the specific use case and problem context. Therefore, there is no universally optimal method for all scenarios; instead, the selection should align with the precise requirements and constraints of the use case.

Future research could explore advanced prompting techniques to improve the performance of ICL, such as Chain-of-Thought (CoT) reasoning or innovative prompt design strategies. These advancements have the potential to further enhance the applicability and efficacy of ICL in various domains.

Appendix

10.1 List of prompts

System Prompt

Only give one best answer without any explanation. The answer should be from the choices given only.

Dataset: CoRT - Data Class

Prompt

In terms of **Data Class Smell**, the following class can be classified as *Smelly* if it contains **Data Class** or *No* if it does not contain the smell. Only answer with the word **Smelly** or the word **No**.
The code sample: `{{ Code Sample Goes Here }}`
The best classified as: `{{ Label Goes Here or leave empty for the test sample }}`

Dataset: CoRT - Feature Envy

Prompt

In terms of **Feature Envy Smell**, the following method can be classified as *Smelly* if it contains **Feature Envy** or *No* if it does not contain the smell. Only answer with the word **Smelly** or the word **No**.
The code sample: `{{ Code Sample Goes Here }}`
The best classified as: `{{ Label Goes Here or leave empty for the test sample }}`

Dataset: CoRT - God Class

Prompt

In terms of **God Class Smell**, the following class can be classified as *Smelly* if it contains **God Class** or *No* if it does not contain the smell. Only answer with the word **Smelly** or the word **No**.
The code sample: `{{ Code Sample Goes Here }}`
The best classified as: `{{ Label Goes Here or leave empty for the test sample }}`

Dataset: CoRT - Long Method

Prompt

In terms of **Long Method Smell**, the following method can be classified as *Smelly* if it contains **Long Method** or *No* if it does not contain the smell. Only answer with the word **Smelly** or the word **No**.
The code sample: `{{ Code Sample Goes Here }}`
The best classified as: `{{ Label Goes Here or leave empty for the test sample }}`

Dataset: MLCQ

Prompt

Classify the following code sample from only one of **(Feature Envy, Blob, Long Method, Data Class)** with the best class.
The code sample: {{ Code Sample Goes Here }}
The best smell classification is: {{ Label Goes Here or leave empty for the test sample }}

Dataset: GPT Dataset

Prompt

Classify the following code sample from only one of **(Long Method, Code Clone, Feature Envy, Data Class, Long Parameter List)** with the best class.
The code sample: {{ Code Sample Goes Here }}
The best smell classification is: {{ Label Goes Here or leave empty for the test sample }}

Artificially Dataset Generation Prompt

Prompt

Generate a code sample from this domain: {{ Domain Goes Here }}, using {{ Language Goes Here }} programming language, that exhibit {{ Smell Goes Here }} code smell. Give the code in a code block.

10.2 Code clone sample forms

The following examples illustrate the two forms of code clone samples used in the artificially constructed dataset.

Code Clone Form 1: Within a Single Class

```
class Product {
    constructor(name, price) {
        this.name = name;
        this.price = price;
    }
    calculateDiscount(discount) {
        return this.price - (this.price * (discount / 100));
    }
    calculateDiscountForVIP(discount) {
        return this.price - (this.price * (discount / 100));
    }
    calculateDiscountForSeasonal(discount) {
        return this.price - (this.price * (discount / 100));
    }
    displayProductInfo() {
        return `${this.name}: $${this.price}`;
    }
    displayProductInfoForVIP() {
        return `${this.name} (VIP): $${this.price}`;
    }
    displayProductInfoForSeasonal() {
        return `${this.name} (Seasonal): $${this.price}`;
    }
}
```

Code Clone Form 2: Between Two Short Classes

```
class Order:
    def __init__(self, order_id, user_id, items):
        self.order_id = order_id
        self.user_id = user_id
        self.items = items
        self.status = "Pending"
    def process_order(self):
        self.status = "Processing"
        print(f"Processing order {self.order_id}")
    def cancel_order(self):
        self.status = "Cancelled"
        print(f"Cancelling order {self.order_id}")
    def ship_order(self):
        self.status = "Shipped"
        print(f"Shipping order {self.order_id}")

class ReturnOrder:
    def __init__(self, return_id, order_id, user_id, items):
        self.return_id = return_id
        self.order_id = order_id
        self.user_id = user_id
        self.items = items
        self.status = "Pending"
    def process_return(self):
        self.status = "Processing"
        print(f"Processing return {self.return_id}")
    def cancel_return(self):
        self.status = "Cancelled"
        print(f"Cancelling return {self.return_id}")
    def ship_return(self):
        self.status = "Shipped"
        print(f"Shipping return {self.return_id}")
```

Acknowledgements The authors acknowledge the support of the King Fahd University of Petroleum and Minerals in the development of this work.

Author Contributions Alomari contributed to the experiment and was the primary author of the manuscript. Redah contributed to the experiment and the writing process. Ashraf contributed to the experiment and authored in the manuscript. Alshayeb supervised the work, reviewed the manuscript, and provided editorial revisions, and edited the manuscript. All authors reviewed and approved the final version.

Funding Funding declaration is not applicable

Data Availibility Statement The execution code and the datasets used can be found in a public GitHub Repository (https://github.com/nawafalomari/ics500/tree/main) (https://github.com/nawafalomari/ics500/tree/main)

Declarations

Conflicts of interest The authors declare that they have no conflict of interest.

Ethical approval Ethical approval declaration is not applicable

Informed consent Informed consent declaration is not applicable

Clinical Trial Number Clinical trial number: not applicable.

References

AbuHassan A, Alshayeb M, Ghouti L (2021) Software smell detection techniques: a systematic literature review. J Softw Evol Process 33(3):e2320

Agarwal R, Singh A, Zhang LM, Bohnet B, Rosias L, Chan SC, Zhang B, Faust A, Larochelle H (2024) Many-shot in-context learning. In: ICML 2024 workshop on in-context learning. Available: https://openreview.net/forum?id=goi7DFHlqS

AI M (2024) Codellama-70b: Meta's language model for coding. available at: https://ai.meta.com

Alazba A, Aljamaan H, Alshayeb M (2023) Deep learning approaches for bad smell detection: a systematic literature review. Empir Softw Eng 28(3):77

Alazba A, Aljamaan H, Alshayeb M (2024) Cort: transformer-based code representations with self-supervision by predicting reserved words for code smell detection. Empir Softw Eng 29(3):59

Alrashedy K, Binjahlan A (2023) Language models are better bug detector through code-pair classification. arXiv preprint arXiv:2311.07957

Alshayeb M (2011) The impact of refactoring to patterns on software quality attributes. Arab J Sci Eng 36:1241–1251

Bafandeh Mayvan B, Rasoolzadegan A, Javan Jafari A (2020) Bad smell detection using quality metrics and refactoring opportunities. J Softw Evol Process 32(8):e2255

Banker RD, Datar SM, Kemerer CF, Zweig D (1993) Software complexity and maintenance costs. Commun ACM 36(11):81–95

Baumgartner N, Iyenghar P, Schoemaker T, Pulvermüller E (2024) Ai-driven refactoring: a pipeline for identifying and correcting data clumps in git repositories. Electronics 13(9):1644

Bhave A, Sinha R (2022) Deep multimodal architecture for detection of long parameter list and switch statements using distilbert. In: 2022 IEEE 22nd international working conference on source code analysis and manipulation (SCAM). IEEE, pp 116–120

Brown TB (2020) Language models are few-shot learners. arXiv preprint arXiv:2005.14165

Byrt T, Bishop J, Carlin JB (1993) Bias, prevalence and kappa. J Clin Epidemiol 46(5):423–429

Chen T (2024) Research of evaluating the effectiveness of large language models in identifying and correcting code anomalies. In: 2024 5th International seminar on artificial intelligence, networking and information technology (AINIT). IEEE, pp 360–363

Chen Z, Chen L, Ma W, Zhou X, Zhou Y, Xu B (2018) Understanding metric-based detectable smells in python software: a comparative study. Inf Softw Technol 94:14–29

Cohen J (1960) A coefficient of agreement for nominal scales. Educ Psychol Measur 20(1):37–46

Cohn DA, Atlas LE, Ladner RE (1994) Improving generalization with active learning. In: 1994 Machine learning proceedings. Elsevier, pp 201–221

Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L (2023) Qlora: efficient finetuning of quantized llms. 2 . arxiv:2305.14314

Diederik PK (2014) Adam: a method for stochastic optimization. (No Title)

Dodge J, Ilharco G, Schwartz R, Farhadi A, Hajishirzi H, Smith N (2020) Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. arXiv preprint arXiv:2002.06305

Elish KO, Alshayeb M (2011) A classification of refactoring methods based on software quality attributes. Arab J Sci Eng 36:1253–1267

Feinstein AR, Cicchetti DV (1990) High agreement but low kappa: I. the problems of two paradoxes. J Clin Epidemiol 43(6):543–549

Feng Z et al (2020) Codebert: A pre-trained model for programming and natural languages. In: Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)

Fontana FA, Ferme V, Marino A, Walter B, Martenka P (2013) Investigating the impact of code smells on system's quality: an empirical study on systems of different application domains. In: 2013 IEEE international conference on software maintenance. IEEE, pp 260–269

Fowler M (2018) Refactoring: improving the design of existing code. Addison-Wesley Professional

Guo D, Ren S, Lu S, Feng Z, Tang D, Liu S, Zhou L, Duan N, Svyatkovskiy A, Fu S et al (2020) Graphcodebert: pre-training code representations with data flow. arXiv:2009.08366

Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: Unified cross-modal pre-training for code representation. arXiv preprint arXiv:2203.03850

Hamdy A, Tazy M (2020) Deep hybrid features for code smells detection. J Theor Appl Inf Technol 98(14):2684–2696

Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. arXiv preprint arXiv:1801.06146
Jedlitschka A, Ciolkowski M, Pfahl D (2008) Reporting experiments in software engineering. In: Guide to advanced empirical software engineering, pp 201–228
Jones KS, Walker S, Robertson SE (2000) A probabilistic model of information retrieval: development and comparative experiments: Part 2. Inf Process Manag 36(6):809–840
Kenton JDM-WC, Toutanova LK (2019) Bert: pre-training of deep bidirectional transformers for language understanding," in Proceedings of naacL-HLT, vol 1. Minneapolis, Minnesota, p 2
Kovačević A, Slivka J, Vidaković D, Grujić K-G, Luburić N, Prokić S, Sladić G (2022) Automatic detection of long method and god class code smells through neural source code embeddings. Expert Syst Appl 204:117607
Lacerda G, Petrillo F, Pimenta M, Guéhéneuc YG (2020) Code smells and refactoring: a tertiary systematic review of challenges and observations. J Syst Softw 167:110610
Li G, Tang Y, Zhang X, Yi B (2020) A deep learning based approach to detect code clones. In: 2020 International Conference on Intelligent Computing and Human-Computer Interaction (ICHCI). IEEE, pp 337–340
Liu H, Jin J, Xu Z, Zou Y, Bu Y, Zhang L (2019) Deep learning based code smell detection. IEEE Trans Softw Eng 47(9):1811–1837
Liu X, Zhang F, Hou Z, Mian L, Wang Z, Zhang J, Tang J (2021) Self-supervised learning: generative or contrastive. IEEE Trans Knowl Data Eng 35(1):857–876
Liu J, Sha C, Peng X (2023) Improving fine-tuning pre-trained models on small source code datasets via variational information bottleneck. In: 2023 IEEE international conference on software analysis, evolution and reengineering (SANER). IEEE, pp 331–342
Lu S, Guo D, Ren S, Huang J, Svyatkovskiy A, Blanco A, Clement C, Drain D, Jiang D, Tang D et al (2021) Codexglue: a machine learning benchmark dataset for code understanding and generation. arXiv preprint arXiv:2102.04664
Madeyski L, Lewowski T (2020) Mlcq: industry-relevant code smell data set. In: Proceedings of the 24th international conference on evaluation and assessment in software engineering, pp 342–347
Madeyski L, Lewowski T (2023) Detecting code smells using industry-relevant data. Inf Softw Technol 155:107112
Matthews BW (1975) Comparison of the predicted and observed secondary structure of t4 phage lysozyme. Biochem Biophys Acta (BBA)-Protein Struct 405(2):442–451
Min S, Lyu X, Holtzman A, Artetxe M, Lewis M, Hajishirzi H, Zettlemoyer L (2022) Rethinking the role of demonstrations: what makes in-context learning work? [Online]. Available: arxiv:2202.12837
Moha N, Guéhéneuc Y-G, Duchien L, Le Meur A-F (2009) Decor: a method for the specification and detection of code and design smells. IEEE Trans Softw Eng 36(1):20–36
Mosbach M, Pimentel T, Ravfogel S, Klakow D, Elazar Y (2023) Few-shot fine-tuning vs. in-context learning: A fair comparison and evaluation. arXiv preprint arXiv:2305.16938
Mueller A, Webson A, Petty J, Linzen T (2023) In-context learning generalizes, but not always robustly: the case of syntax. arXiv preprint arXiv:2311.07811
Ouyang S, Zhang JM, Harman M, Wang M (2024) An empirical study of the non-determinism of chatgpt in code generation. ACM Transactions on Software Engineering and Methodology. https://doi.org/10.1145/3697010
Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I et al (2019) Language models are unsupervised multitask learners. OpenAI blog 1(8):9
Ren S, Shi C, Zhao S (2021) Exploiting multi-aspect interactions for god class detection with dataset fine-tuning. In: 2021 IEEE 45th annual computers, software, and applications conference (COMPSAC). IEEE, pp 864–873
Renze M, Guven E (2024) The effect of sampling temperature on problem solving in large language models. Available: arxiv:2402.05201
Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T et al (2023) Code llama: open foundation models for code. arXiv preprint arXiv:2308.12950
Sharma K, Chhabra JK (2025) Thresholds derivation of software code metrics for god class detection using metaheuristic approaches
Sharma T, Efstathiou V, Louridas P, Spinellis D (2021) Code smell detection by deep direct-learning and transfer-learning. J Syst Softw 176:110936
Silva LL, Silva J, Montandon JE, Andrade M, Valente MT (2024) Detecting code smells using chatgpt: initial insights. In: Proceedings of the 18th ACM/IEEE international symposium on empirical software engineering and measurement, pp 400–406

Škipina M, Slivka J, Luburić N, Kovačević A (2024) Automatic detection of feature envy and data class code smells using machine learning. Expert Syst Appl 243:122855. https://www.sciencedirect.com/science/article/pii/S0957417423033572

Team G, Anil R, Borgeaud S, Alayrac J-B, Yu J, Soricut R, Schalkwyk J, Dai AM, Hauth A, Millican K et al (2023) Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805

Wang H, Liu J, Kang J, Yin W, Sun H, Wang H (2020) Feature envy detection based on bi-lstm with self-attention mechanism. In: 2020 IEEE intl conf on parallel & distributed processing with applications, big data & cloud computing, sustainable computing & communications, social computing & networking (ISPA/BDCloud/SocialCom/SustainCom). IEEE, pp 448–457

Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24 824-24 837

Wu L, Zhao Y, Hou X, Liu T, Wang H (2024) Chatgpt chats decoded: Uncovering prompt patterns for superior solutions in software development lifecycle. In: 2024 IEEE/ACM 21st international conference on mining software repositories (MSR). IEEE, pp 142–146

Yamashita A, Counsell S (2013) Code smells as system-level indicators of maintainability: an empirical study. J Syst Softw 86(10):2639–2653

Yin Q, He X, Deng L, Leong CT, Wang F, Yan Y, Shen X, Zhang Q (2024) Deeper insights without updates: The power of in-context learning over fine-tuning. arXiv preprint arXiv:2410.04691

Zhang Z, Saber T (2024) Assessing the code clone detection capability of large language models. In: 2024 4th International conference on code quality (ICCQ). IEEE, pp 75–83

Zhang D, Song S, Zhang Y, Liu H, Shen G (2023) Code smell detection research based on pre-training and stacking models. IEEE Lat Am Trans 22(1):22–30

Zhang Z, Chen C, Liu B, Liao C, Gong Z, Yu H, Li J, Wang R (2023) A survey on language models for code. arXiv preprint arXiv:2311.07989

Zhang X, Lv A, Liu Y, Sung F, Liu W, Luan J, Shang S, Chen X, Yan R (2025) More is not always better? Enhancing many-shot in-context learning with differentiated and reweighting objectives. arXiv preprint arXiv:2501.04070

Authors and Affiliations

Nawaf Alomari[1] · Moussa Redah[1] · Ahmad Ashraf[1] · Mohammad Alshayeb[1,2]

✉ Nawaf Alomari
g201931050@kfupm.edu.sa

Moussa Redah
g202392670@kfupm.edu.sa

Ahmad Ashraf
g202411740@kfupm.edu.sa

Mohammad Alshayeb
alshayeb@kfupm.edu.sa

[1] Information and Computer Science Department, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia

[2] Interdisciplinary Research Center forIntelligent Secure Systems, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia